General Sir John Kotelawala Defence University

Web Development Group Assignment

Project Report

Online Book & Educational Donation Management System

Group No.	05
Group Members	K.D. Chamathsara W.M.D.A. Senarathna H.R.D.N. Bandara R.D.T.S. Senadeera
Technologies	HTML, CSS, JavaScript, PHP, MySQL
Submission	October 2026

Table of Contents

Table of Contents 1
Group Information 2
1. Project overview 3
1.1 Introduction 3
1.2 Problem statement 4
1.3 Objectives 4
1.4 System modules and stakeholders 4
Key stakeholders 4
1.5 System architecture and workflow 5
Donation process 5
2.Tools and technologies 6
2.1 Frontend and backend connection 6
2.2 Technology restrictions 6
2.3 Responsive design and access controls 7
Responsive web design 7
Implemented access and data controls 7
3. DATABASE DESIGN 8
3.1 Core entity relationship diagram 8
3.2 Extended database relationships 9
3.3 Main database schema 10
Relationships and deletion behaviour 10
3.4 Verification and supporting schema 11
3.5 Implemented database evidence 12
3.6 Implemented table structures 13
4. USER INTERFACES 14
5. Future enhancements 35
5.1 Improvements to the current system 35
5.2 Additional features 35
5.3 Commercialization and scalability 35
6. Deployment and submission preparation 36
6.1 Local installation 36
6.2 Verification performed for this report 36
6.3 Submission checklist 36
7. Conclusion and source references 37

Group Information

Index No.	Name	Degree Programme	Individual Contribution
D/BCS/26/0003	K.D. Chamathsara	Computer Science	Donor Management (registration, profile, add donation, my donations); database design and shared back-end core (configuration, sessions, authentication)
D/COE/26/0013	W.M.D.A. Senarathna	Computer Engineering	Recipient Management: institution registration, institution profile, requirements and claims.
D/BSE/26/0023	H.R.D.N. Bandara	Software Engineering	Administration and Requests: admin dashboard, institution verification, request management and reports.
7371	R.D.T.S. Senadeera	Software Engineering	Login pages, Home page, Navigation pages.

1. Project overview

1.1 Introduction

EduShare is a web application that connects donors of books and educational materials with schools, Piriven and other educational organizations. Donors publish available items, while approved institutions browse donations, submit requests and describe their own requirements. Administrators verify institutions and supervise system activity.

1.2 Problem statement

Usable textbooks, stationery and educational equipment may remain unused while institutions need those resources. Informal donation arrangements make it difficult to locate suitable recipients, identify urgent requirements, verify institutions and maintain reliable records of quantities promised and received. EduShare provides a shared platform for these activities.

1.3 Objectives

• Provide role-based access for donors, institutions and administrators.

• Allow donors to publish and maintain educational item listings with quantities, condition and collection details.

• Allow verified institutions to request available items and publish normal or urgent requirements.

• Help donors find suitable institutions using category, subject, grade, quantity and location filters.

• Support partial quantities, acceptance, delivery confirmation and database-backed notifications.

• Provide administrators with verification, moderation, reports and activity review functions.

1.4 System modules and stakeholders

Module	Implemented responsibility
Public website	Home, About, How it works, donation browsing, urgent needs and contact form.
Authentication and profiles	Role selection at login, member registration, sessions, password checks and account/profile updates.
Institution verification	Registration number, supporting document upload, approval, rejection and requests for more information.

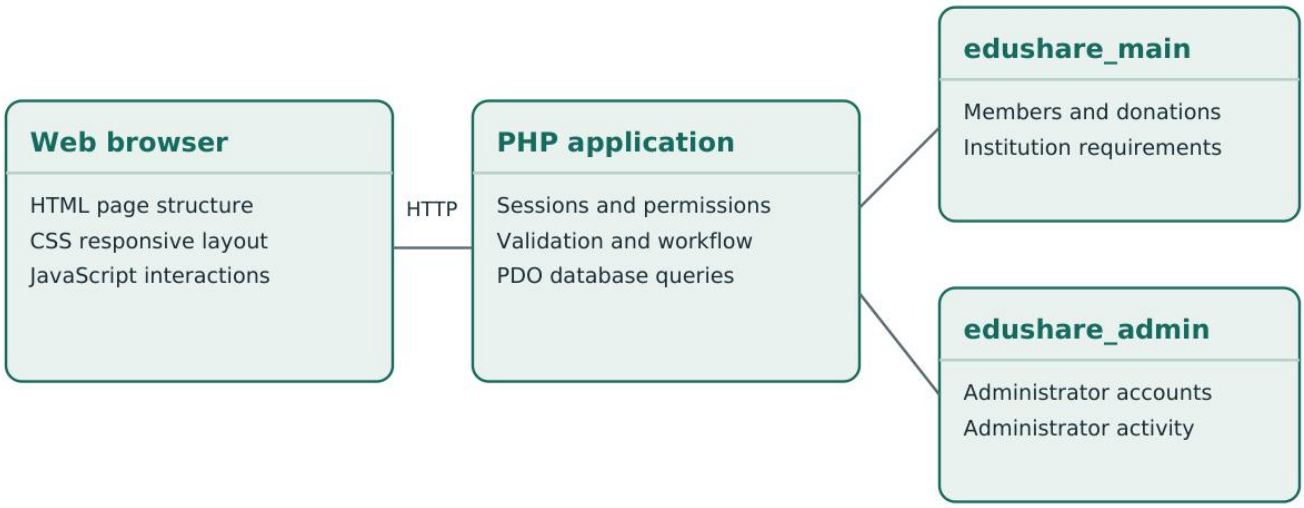
Donation listings	Add, edit and view educational items with category, quantity, condition, image and delivery details.
Requirements and matching	Institution requirements, urgency review, location filters and rule-based matching scores.
Offers and claims	Recipient claims and donor offers share the requests table; accepted quantities reserve stock.
Delivery workflow and impact	Accepted, preparing, ready, delivered and completed stages; receipt confirmation and donor impact.
Administration	Users, categories, listings, requests, urgent review, reports, activity review and contact inbox.

Key stakeholders

User	Main interests
Donor	Find an appropriate institution, manage listings and see completed donation impact.
Institution representative	Obtain required learning materials, view application status and confirm receipt.
Administrator	Check institutions, moderate suspicious activity and supervise accurate system records.
Public visitor	Understand the service, view available donations and approved urgent needs.

1.5 System architecture and workflow

The browser requests PHP pages and uses JavaScript for interactive behaviour. PHP validates actions, checks user permissions and reads or writes the main or administrator database through PHP data objects.



Donation process

1. An institution applies and an administrator reviews its verification information.

2. A donor publishes an item; an approved institution claims it, or a donor offers it against a requirement.

3. The responsible donor or institution accepts and checks available quantities and reserves the accepted amount.

4. The donor records preparing, ready and delivered stages.

5. The institution confirms receipt. The request becomes completed, the requirement received quantity increases and donor impact totals update.

2.Tools and technologies

Technology	Use in EduShare
HTML	Semantic page structure, links, forms, tables and inputs rendered by PHP.
CSS	Shared visual design in assets/css/style.css, grids, Flexbox, cards, badges and responsive breakpoints.
JavaScript	assets/js/app.js handles the mobile menu, form behaviour, image preview, live item search and notification polling.
PHP	Authentication, server-side validation, uploaded file handling, access checks, reports and donation logic.
MySQL	Relational data storage using tables, primary keys, foreign keys, unique constraints and transactions.

2.1 Frontend and backend connection

Forms send POST requests to PHP scripts. For example, donor/add_item.php validates the donation form, inserts a row into items and updates its subject, location and delivery fields. Shared helper functions provide database connections, sessions, escaping and notifications. api/search.php returns HTML fragments for live search; api/notifications.php returns notification counts for JavaScript polling.

2.2 Technology restrictions

The supplied code uses the required HTML, CSS, JavaScript, PHP and MySQL stack without an application framework. Page headers also request Google Fonts.

2.3 Responsive design and access controls

Responsive web design

Every main layout includes a viewport meta tag. CSS Grid and Flexbox arrange forms, cards and navigation. At a maximum width of 900 px, the sidebar moves off-screen and a menu button toggles the open class. The hero and authentication layouts become a single column, two-column forms collapse and padding is reduced. At 700 px, public navigation and action buttons wrap. Tables are placed in horizontally scrollable wrappers.

Flexible card layouts and clamp-based headings reduce fixed sizing. The stylesheet also respects prefers-reduced-motion by disabling transition effects. A captured 390 px donor dashboard is included with the interface evidence.

Implemented access and data controls

Control	Source behaviour
Login and role checks	password_hash and password_verify protect password storage and comparison. Member and administrator sessions use separate keys; protected pages require the correct role.
Session and form protection	Strict session mode, HttpOnly and SameSite cookies, session ID regeneration on login and CSRF tokens on state-changing forms.
Validation and SQL	Parameterized PHP data objects queries are used for input-based operations. PHP checks roles, account status, required fields and quantity limits. HTML output is escaped through e().
Document and image uploads	Institution documents allow PDF/JPG/PNG up to 5 MB and use random names in a private directory. Item/profile images accept supported image types up to 3 MB.
Stock consistency	Acceptance and completion use transactions and row locks so allocated quantities are checked against current item and requirement totals.

3. DATABASE DESIGN

3.1 Core entity relationship diagram



3.3 Main database schema

Table	Important fields and design
users	id INT PK; name VARCHAR(100); email VARCHAR(120) UNIQUE; password VARCHAR(255); role ENUM(recipient, donor); status ENUM(active, blocked); phone, address, institution; province, district, city; latitude and longitude DECIMAL(10,7); profile_image; created_at.
categories	id INT PK; name VARCHAR(80) UNIQUE. Category inserts are reference data rather than donated-item records.
items	id INT PK; donor_id and category_id INT FKs; item_type ENUM(book, education_item); title VARCHAR(160); brand VARCHAR(120); quantity INT; description TEXT; condition_status ENUM(New, Good, Fair); type ENUM(donate, exchange); image VARCHAR(80); status ENUM(pending, approved, reserved, donated, rejected, cancelled); created_at.
items extended fields	subject and education_level VARCHAR(100); location VARCHAR(150); latitude and longitude DECIMAL(10,7); delivery_method VARCHAR(100); availability_date DATE.
requests	id INT PK; item_id and recipient_id INT FKs; message VARCHAR(255); status ENUM(pending, approved, rejected, completed, cancelled); quantity INT; requirement_id INT NULL; source ENUM(claim, offer); response_note VARCHAR(255); stage ENUM(accepted, preparing, ready, delivered, completed); created_at and updated_at.
notifications	id INT PK; user_id INT FK; message VARCHAR(255); link VARCHAR(160); is_read TINYINT(1); created_at.

Relationships and deletion behaviour

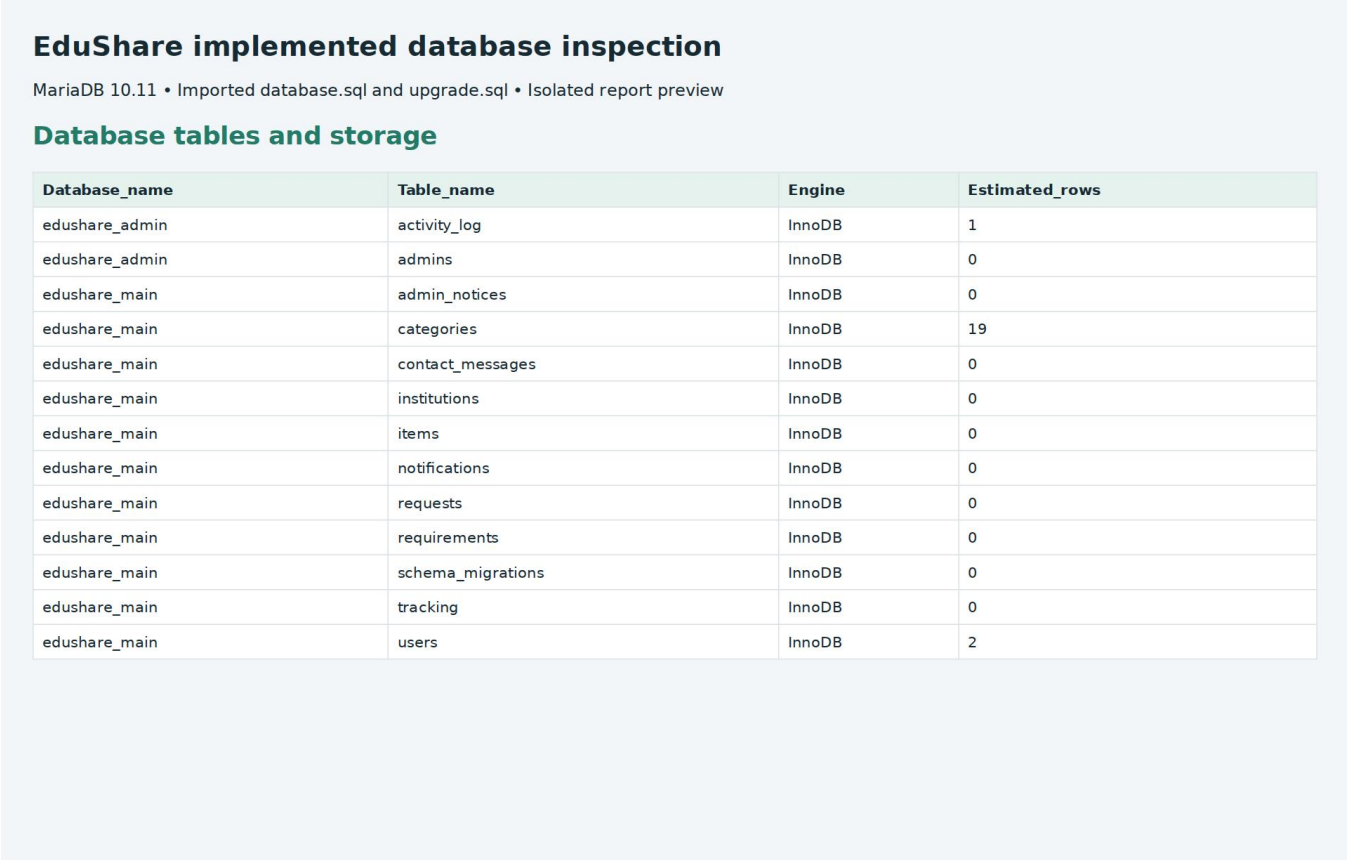
Items and notifications reference users with ON DELETE CASCADE. Requests reference their item and recipient with ON DELETE CASCADE. Categories remain shared reference rows. Requirements reference users and categories without cascading deletion. The admin activity log references its administrator with ON DELETE CASCADE.

3.4 Verification and supporting schema

Table	Important fields and design
institutions	user_id INT PK/FK; institution_type ENUM(School, Piriven, Educational Organization, Other); registration_number VARCHAR(100) UNIQUE; contact_person VARCHAR(100); description TEXT; verification_document VARCHAR(80); verification_status ENUM(pending, approved, rejected, more_information); review_note TEXT; verified_by INT; verified_at DATETIME.
requirements	id INT PK; recipient_id and category_id INT FKs; title VARCHAR(160); subject and education_level VARCHAR(100); quantity_required and quantity_received INT; description and reason TEXT; required_date DATE; urgency ENUM(normal, high); urgency_status ENUM(not_required, pending, approved, rejected); status ENUM(open, completed, cancelled); created_at.
tracking	id INT PK; request_id INT FK; status VARCHAR(40); actor VARCHAR(100); remarks VARCHAR(255); created_at. It stores manually recorded workflow events.
admin_notices	id INT PK; message VARCHAR(255); is_read TINYINT; created_at. These administrator notifications live in the main database.
contact_messages	id INT PK; name VARCHAR(100); email VARCHAR(120); message TEXT; created_at. Stores public contact submissions.
schema_migrations	version VARCHAR(80) PK; applied_at TIMESTAMP. Records the one-time partial quantity migration.
admins	In edushare_admin: id INT PK; name VARCHAR(100); email VARCHAR(120) UNIQUE; password VARCHAR(255); created_at.
activity_log	In edushare_admin: id INT PK; admin_id INT FK; action VARCHAR(255); created_at.

3.5 Implemented database evidence

The schema was imported successfully into a local MariaDB 10.11 instance. The view below queries information_schema.TABLES and shows the main and administrator databases, their actual tables and InnoDB engines.



EduShare implemented database inspection

MariaDB 10.11 • Imported database.sql and upgrade.sql • Isolated report preview

Database tables and storage

Database_name	Table_name	Engine	Estimated_rows
edushare_admin	activity_log	InnoDB	1
edushare_admin	admins	InnoDB	0
edushare_main	admin_notices	InnoDB	0
edushare_main	categories	InnoDB	19
edushare_main	contact_messages	InnoDB	0
edushare_main	institutions	InnoDB	0
edushare_main	items	InnoDB	0
edushare_main	notifications	InnoDB	0
edushare_main	requests	InnoDB	0
edushare_main	requirements	InnoDB	0
edushare_main	schema_migrations	InnoDB	0
edushare_main	tracking	InnoDB	0
edushare_main	users	InnoDB	2

Figure 1. Implemented database table list from the isolated database inspector.

3.6 Implemented table structures

EduShare implemented database inspection

MariaDB 10.11 • Imported database.sql and upgrade.sql • Isolated report preview

edushare_main.items

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI		auto_increment
donor_id	int(11)	NO	MUL		
category_id	int(11)	NO	MUL		
item_type	enum('book','education_item')	NO		book	
title	varchar(160)	NO			
brand	varchar(120)	YES			
quantity	int(11)	NO		1	
description	text	YES			
condition_status	enum('New','Good','Fair')	NO		Good	
type	enum('donate','exchange')	NO		donate	
image	varchar(80)	YES			
status	enum('pending','approved','reserved','donated','rejected','cancelled')	NO		pending	
created_at	timestamp	YES		current_timestamp()	
subject	varchar(100)	YES			
education_level	varchar(100)	YES			
location	varchar(150)	YES			
latitude	decimal(10,7)	YES			
longitude	decimal(10,7)	YES			
delivery_method	varchar(100)	YES			
availability_date	date	YES			

Figure 2. DESCRIBE edushare_main.items showing column types, nullability, keys and defaults.

EduShare implemented database inspection

MariaDB 10.11 • Imported database.sql and upgrade.sql • Isolated report preview

edushare_main.requests

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI		auto_increment
item_id	int(11)	NO	MUL		
recipient_id	int(11)	NO	MUL		
message	varchar(255)	YES			
status	enum('pending','approved','rejected','completed','cancelled')	NO		pending	
created_at	timestamp	YES		current_timestamp()	
updated_at	timestamp	YES		current_timestamp()	on update current_timestamp()
quantity	int(11)	NO		1	
requirement_id	int(11)	YES			
source	enum('claim','offer')	NO		claim	
response_note	varchar(255)	YES			
stage	enum('accepted','preparing','ready','delivered','completed')	NO		accepted	

Figure 3. DESCRIBE edushare_main.requests including quantity, source, requirement_id and stage.

4. USER INTERFACES

Public home and introduction

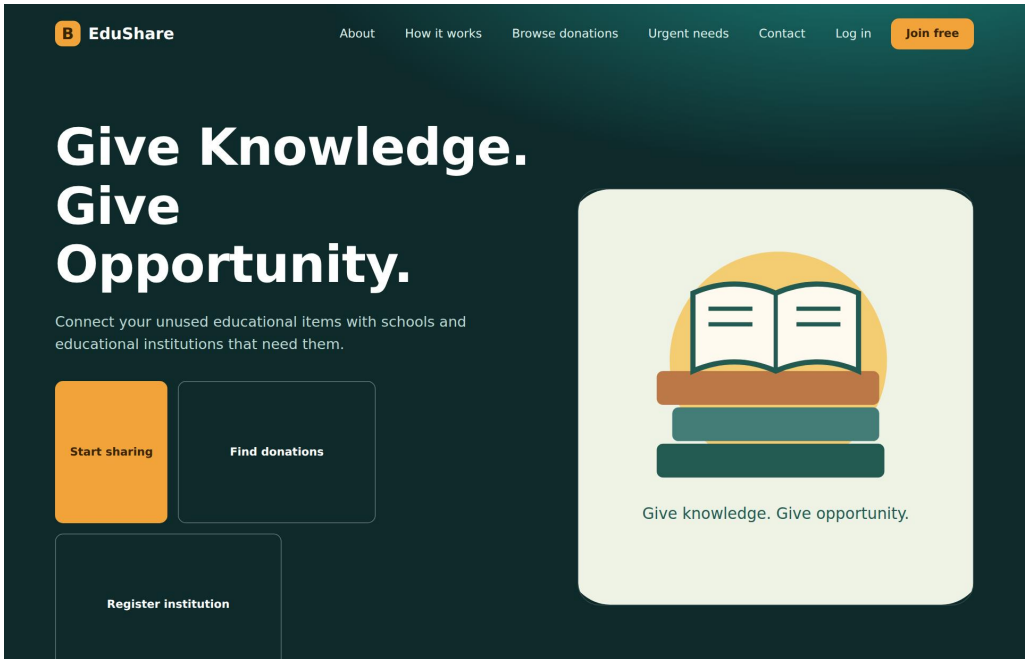


Figure 4. Home page with the EduShare identity and donation entry points.

The home page presents the purpose of EduShare and links to registration, donation browsing and urgent needs. Its lower sections obtain counters, recent listings and urgent requirements from the database.

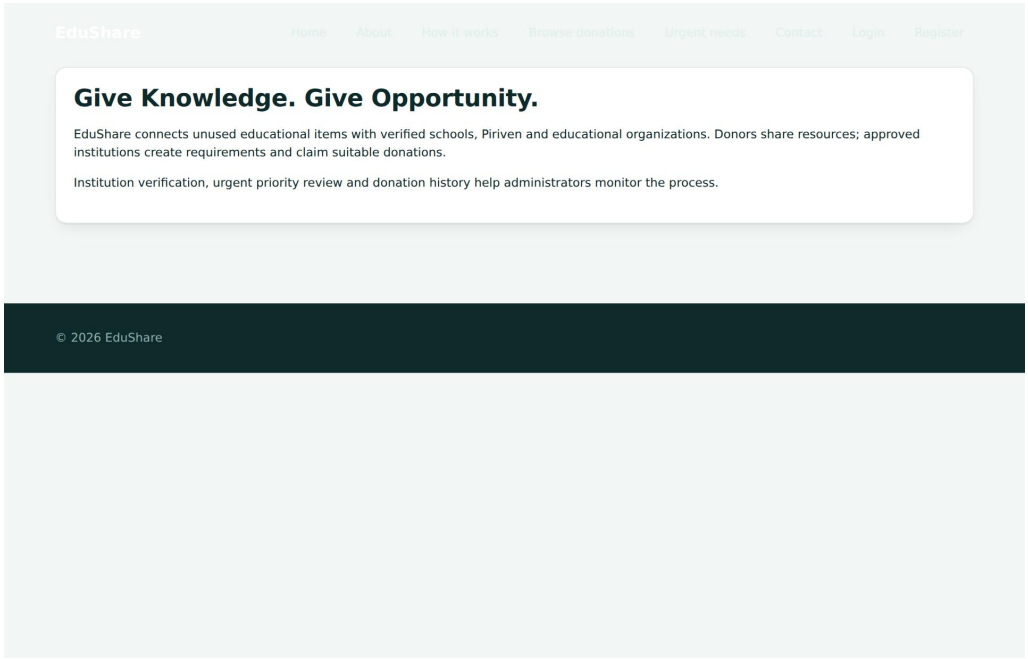


Figure 5. About page explaining the platform.

The About interface explains how donors and verified institutions share educational resources and why verification and quantity records are needed.

Public guidance and contact

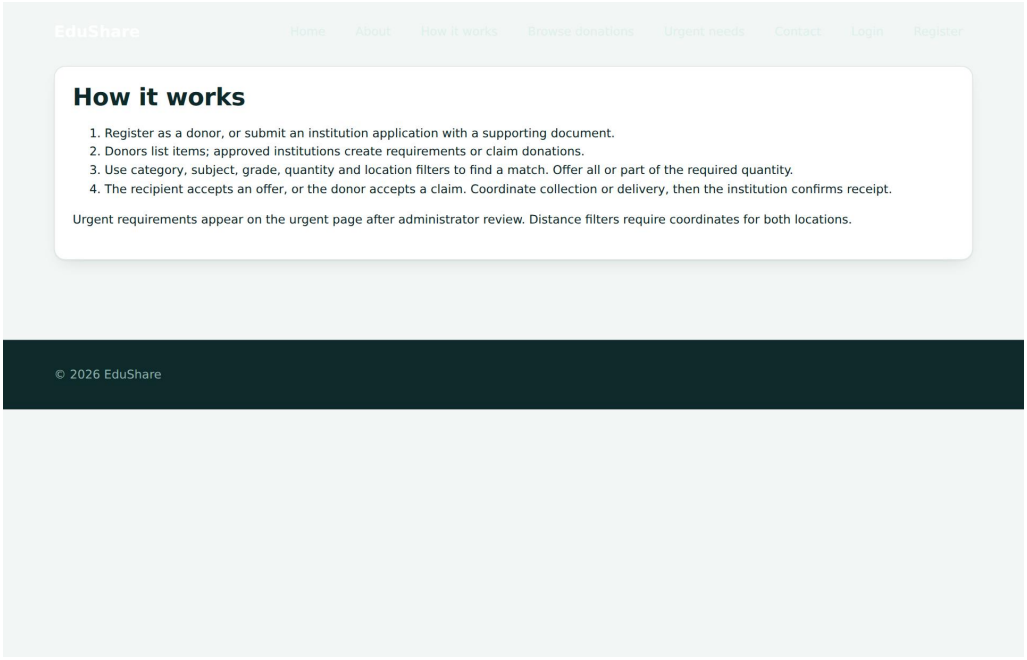


Figure 6. How it works page.

This page describes the registration, verification, listing, requesting and handover sequence for visitors.

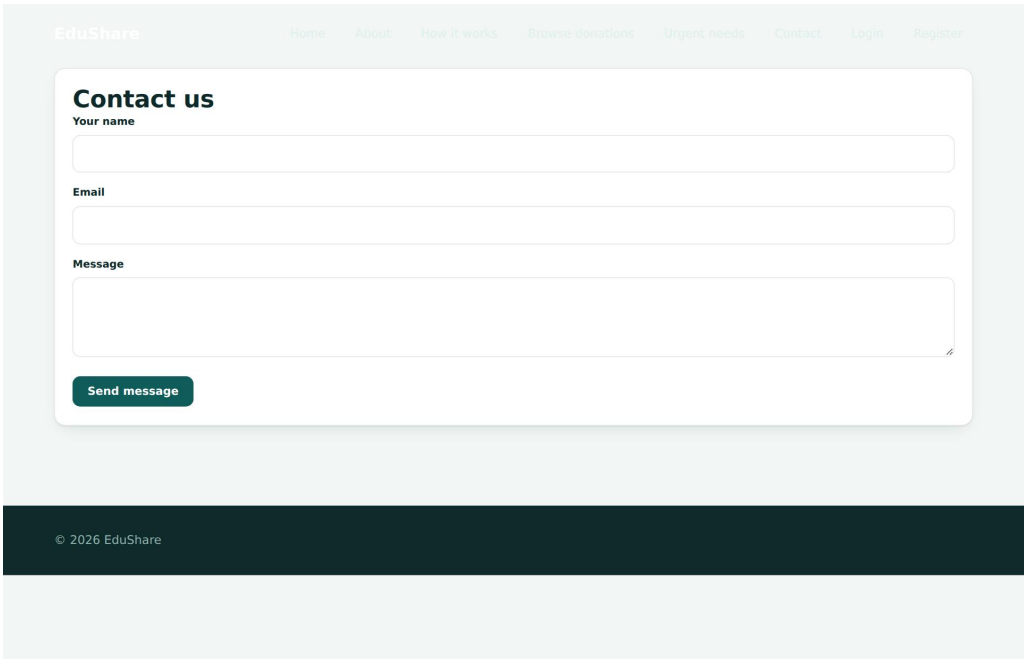


Figure 7. Contact form.

Visitors can submit a name, email and message. PHP validates the submission and saves it in contact_messages for the administrator inbox.

Public donations and urgent needs

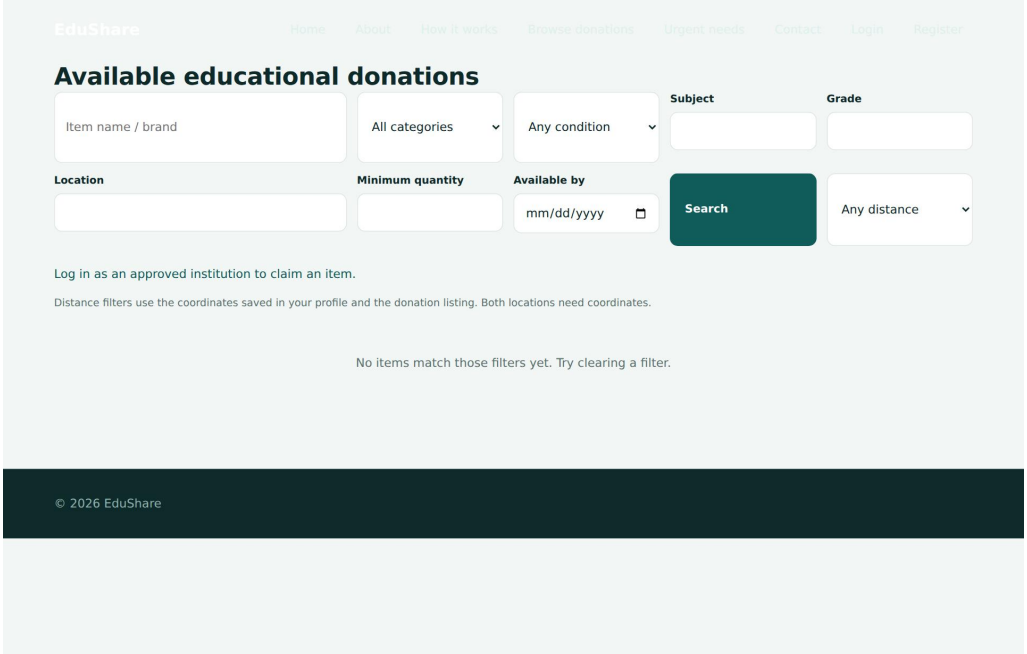


Figure 8. Public donation browser.

Visitors filter educational donations by title, category, condition, subject, level and location. A user signs in before claiming an item.

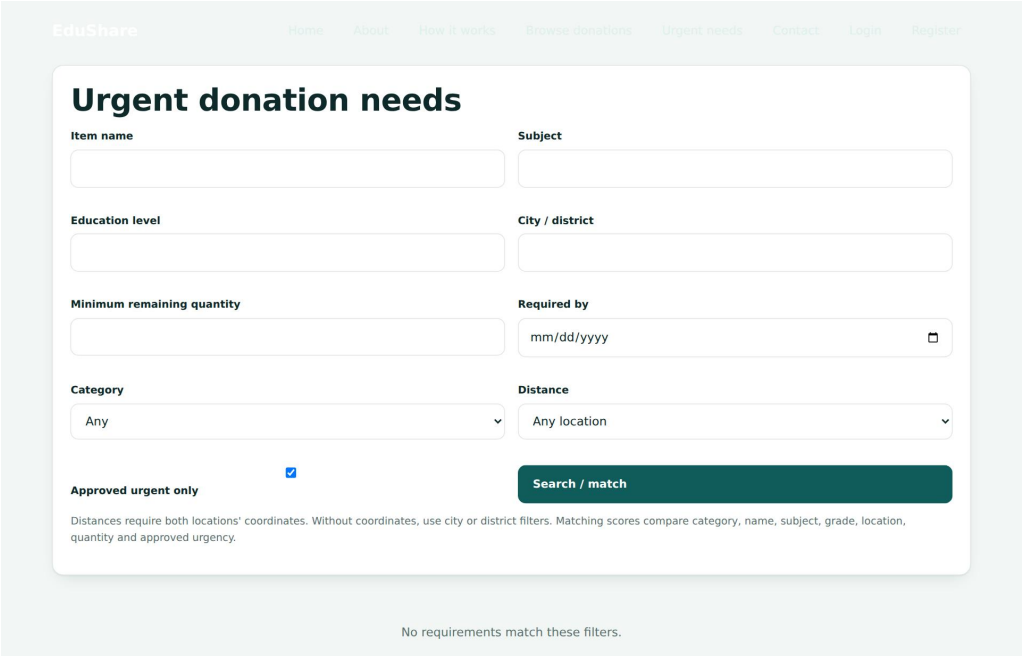


Figure 9. Approved urgent requirements browser.

The urgent view lists open requirements from approved institutions after administrator urgency approval. The preview contains no requirement records, so the results are empty.

Login and administrator access

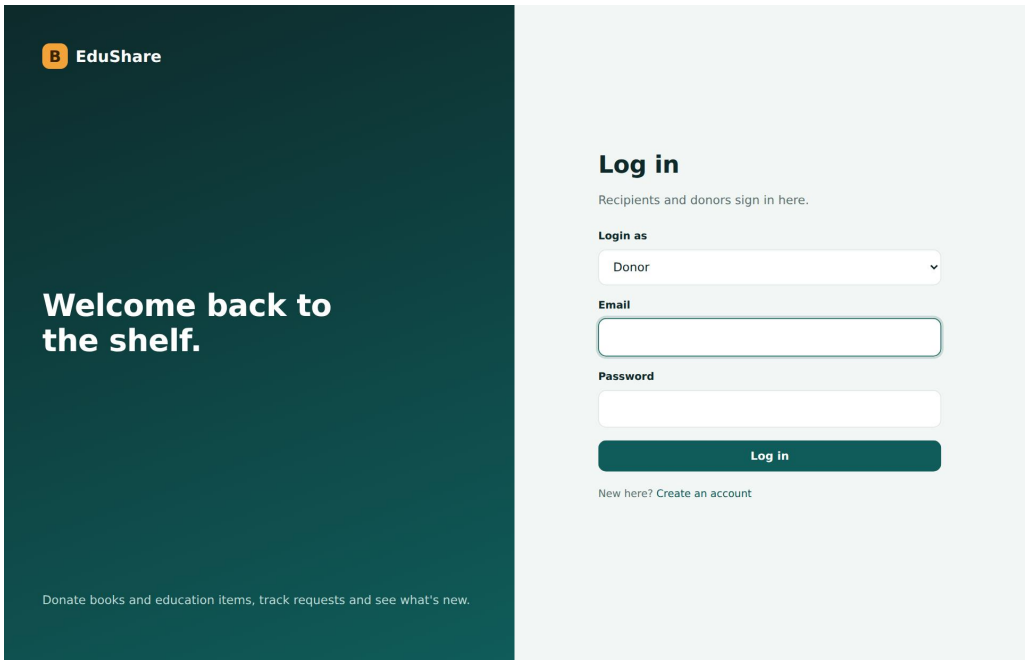


Figure 10. Shared login page with Donor selected.

Users choose Donor, Verified institution or Administrator and enter an email and password. PHP checks credentials in the appropriate database and redirects to the correct portal.

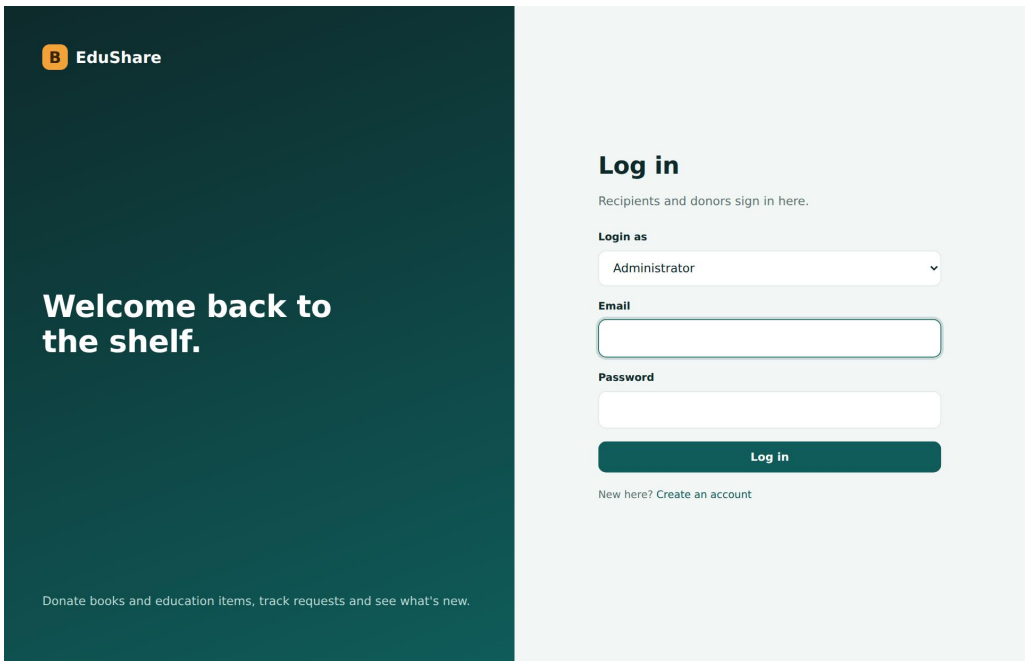


Figure 11. Same login page with Administrator selected.

Administrator access requires an existing admins record and a valid password. Public registration does not create administrator accounts; setup.php provides initial administrator creation.

Donor account registration

The donor role is selected on the registration page. Name, email and password are validated, password confirmation is checked and a donor row is stored in users. Location and an optional image can be supplied.

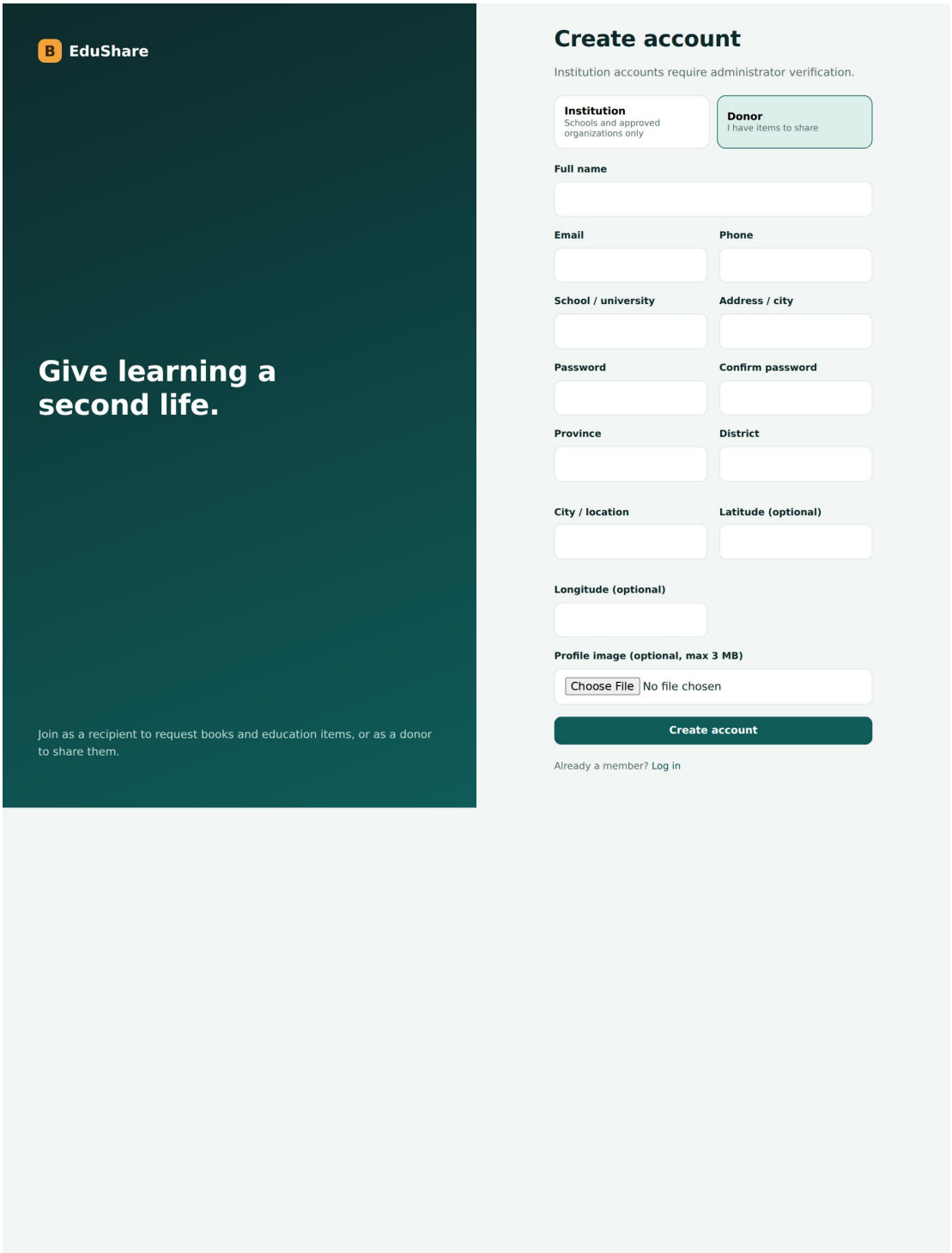


Figure 12. Donor registration form.

Institution account registration

Institution applicants supply their institution name, contact details, address, location, unique registration number and a supporting PDF/JPG/PNG. The initial verification state is pending, and an administrator must approve access to institution functions.

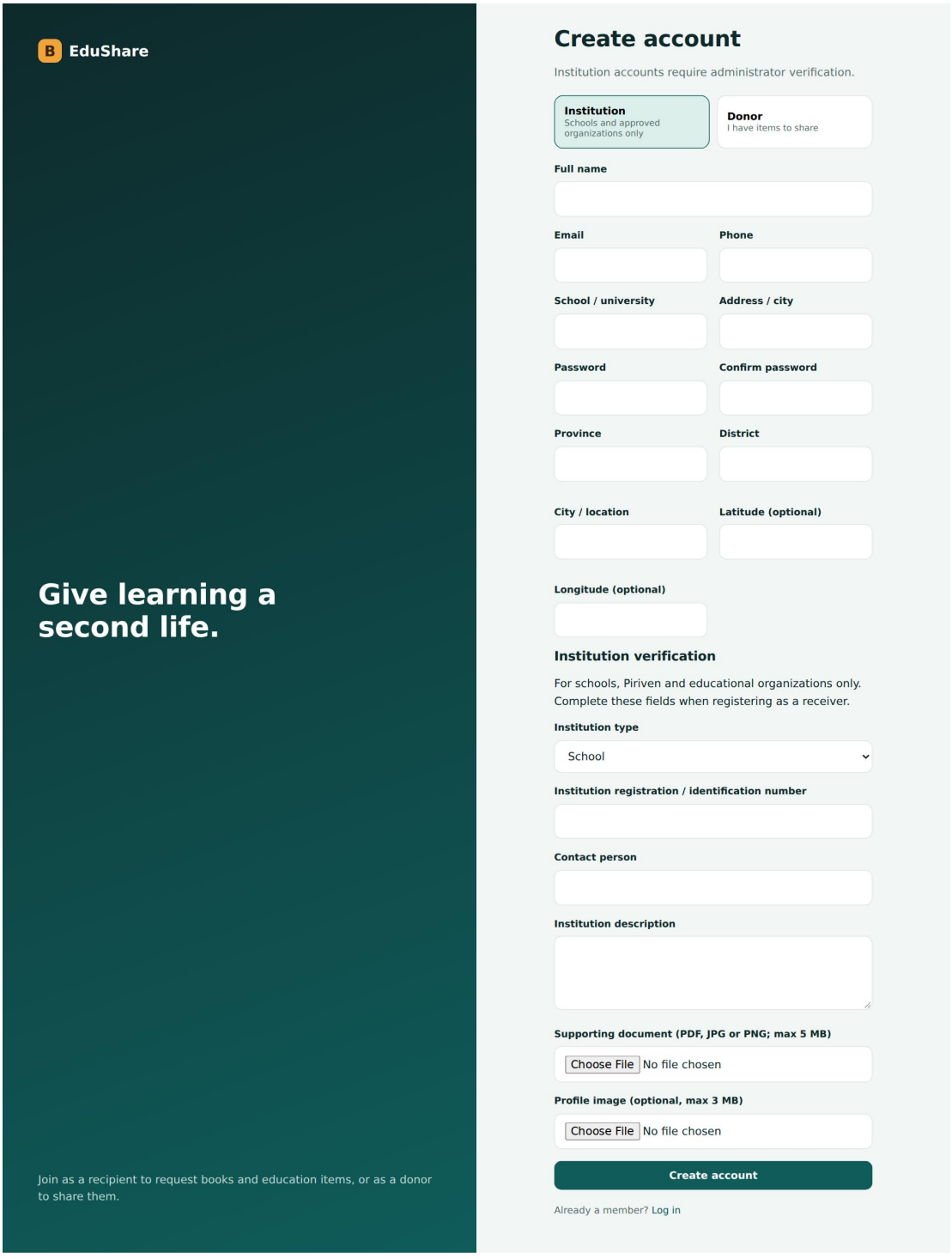


Figure 13. Institution registration form including verification fields.

Donor dashboard and listings

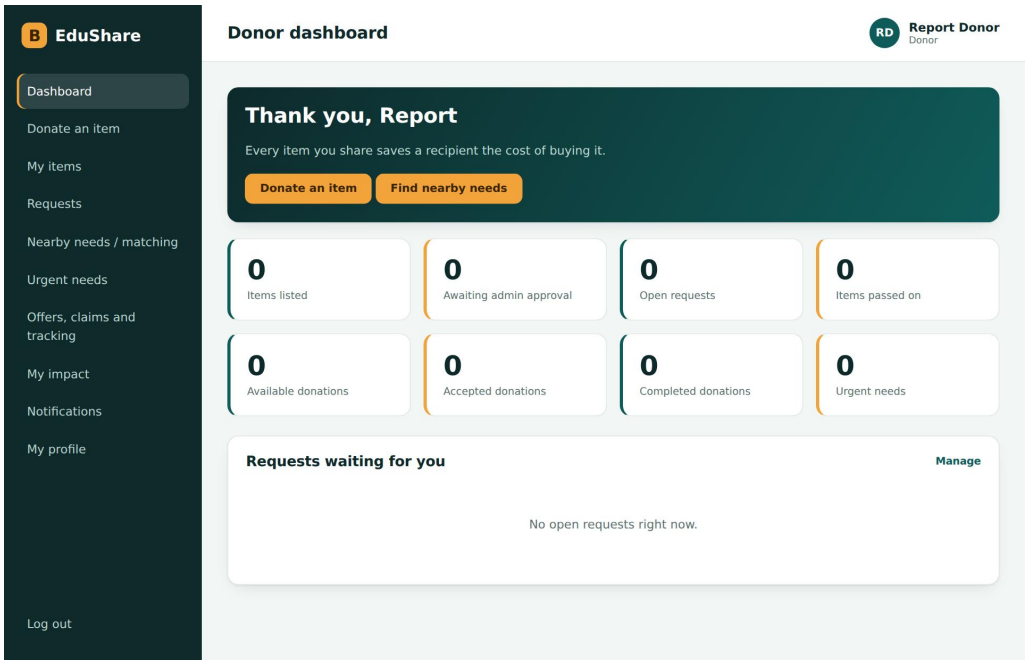


Figure 14. Donor dashboard.

Database queries summarize the donor's listings, pending requests, accepted donations and completed donations. Links lead to adding an item and finding nearby institution needs.

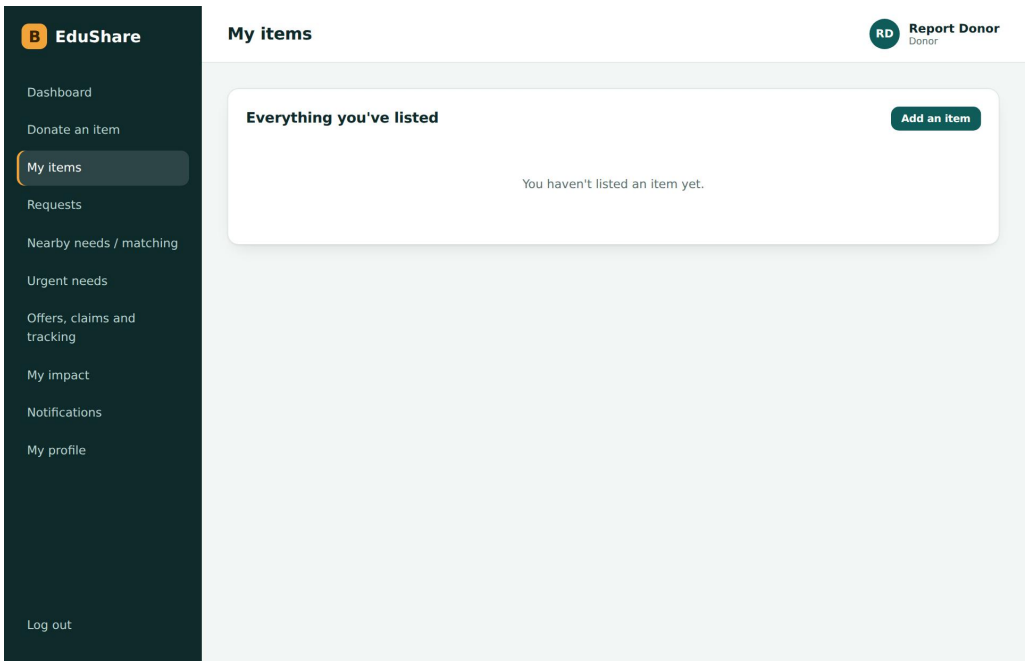


Figure 15. Donor My items interface.

This page shows listings owned by the logged-in donor. Existing records provide editing and status-related actions; no listings exist in the isolated capture database.

Donation entry form

The donor supplies an item name, type, category, condition, positive quantity, location and collection or delivery method. Images, subject, grade, coordinates and availability date add useful context. Normal donations become available immediately after a successful submission.

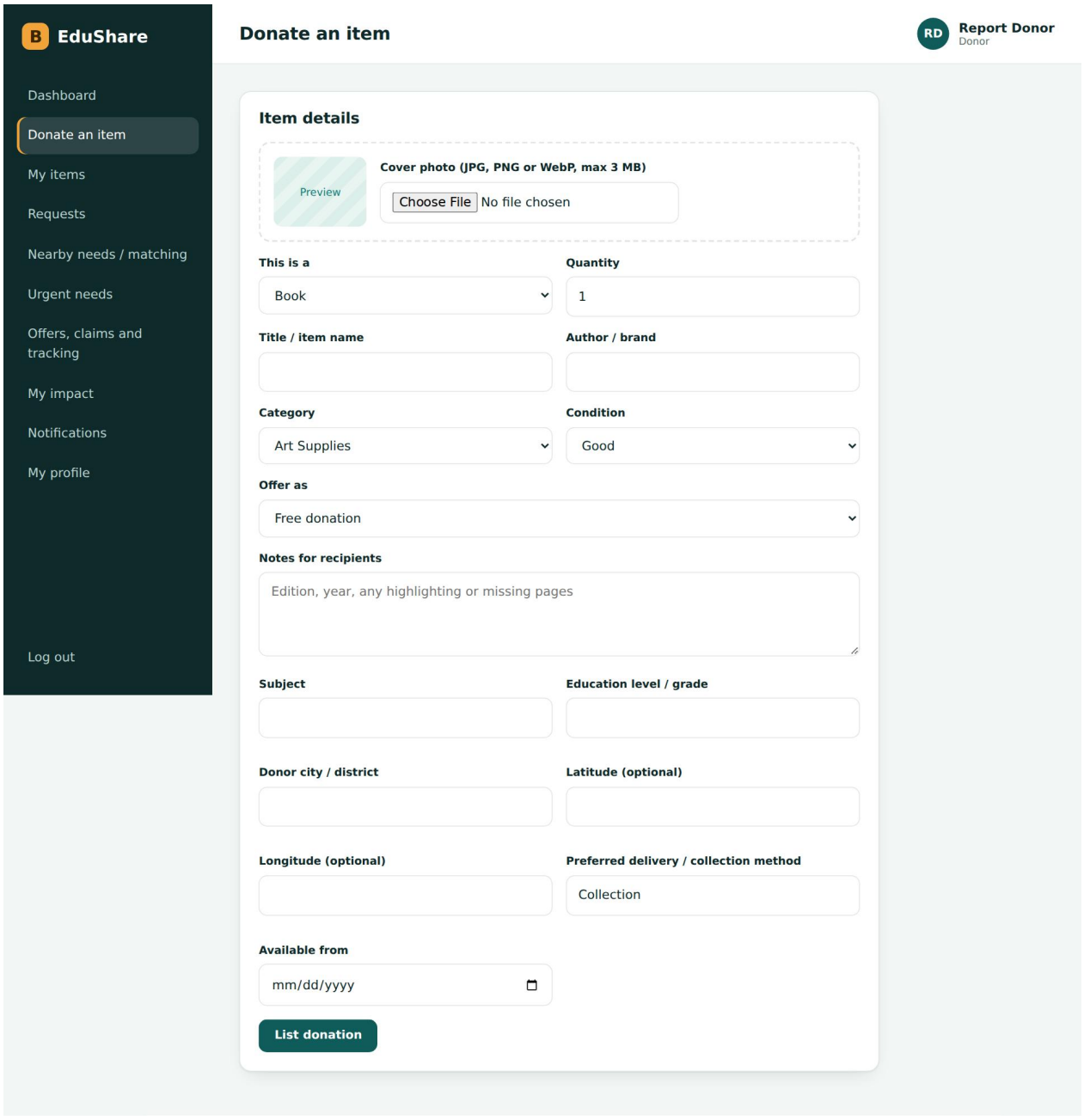


Figure 16. Donate an item form.

Matching and donor impact

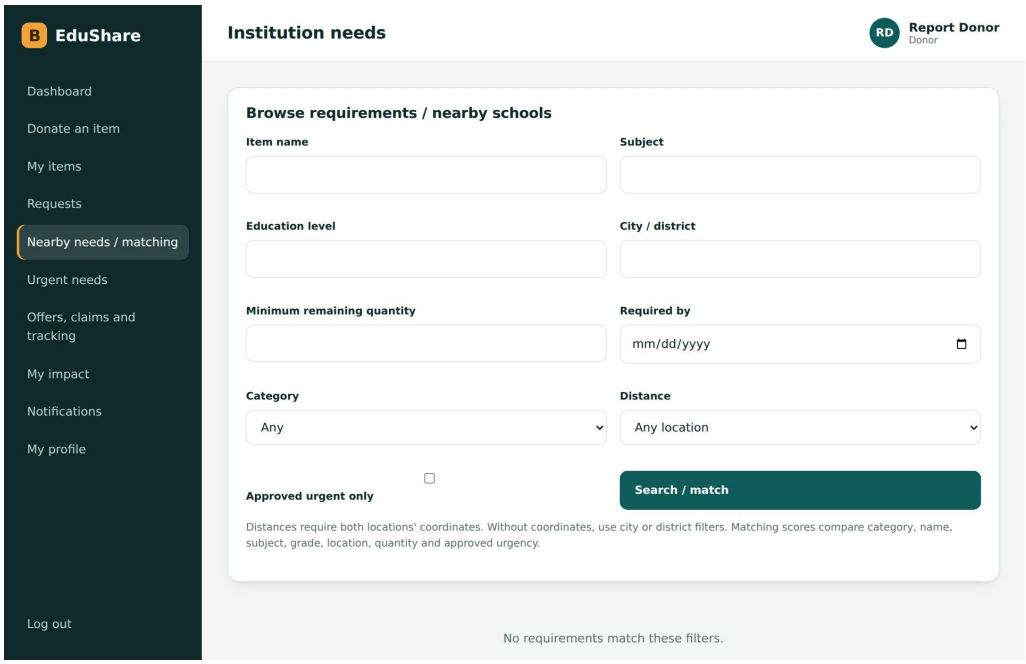


Figure 17. Nearby needs and matching interface.

Donors search by category, subject, grade, minimum quantity, deadline and location. With a selected donation, the PHP matching score compares these attributes and approved urgency. Distance filters require coordinates for both locations.

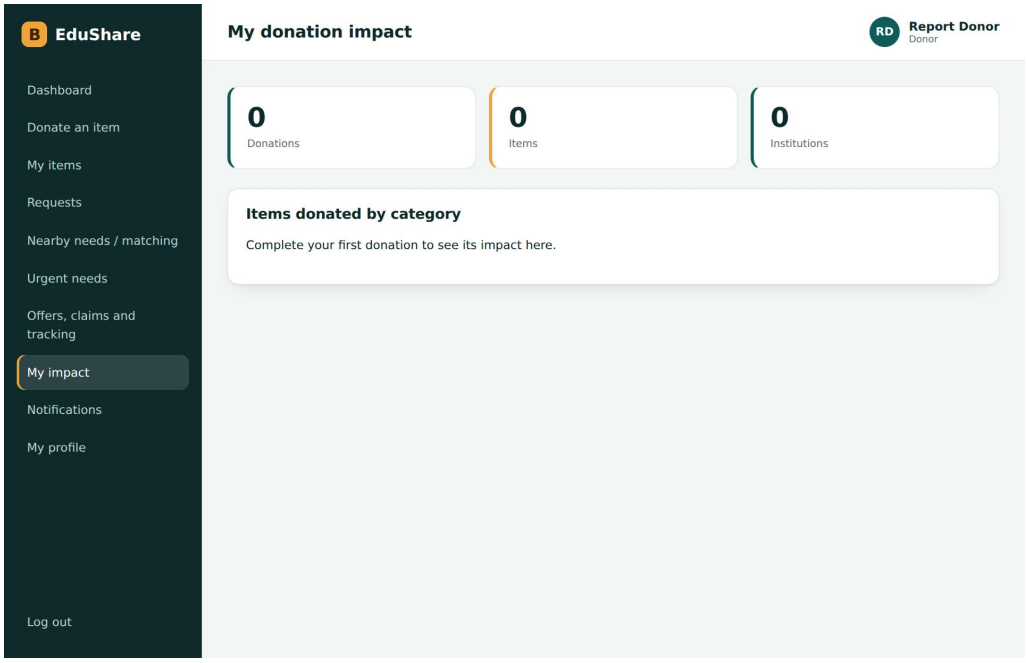


Figure 18. Donor impact interface.

Impact summaries and category totals are calculated from completed requests and their quantities, rather than hard-coded statistics. The empty preview has no completed donations.

Offers claims and delivery workflow

Donor requests.php and recipient requests.php redirect to this shared workflow. Claims and offers can be accepted or rejected by the responsible party. Accepted quantities progress through preparing, ready and delivered, followed by recipient confirmation.

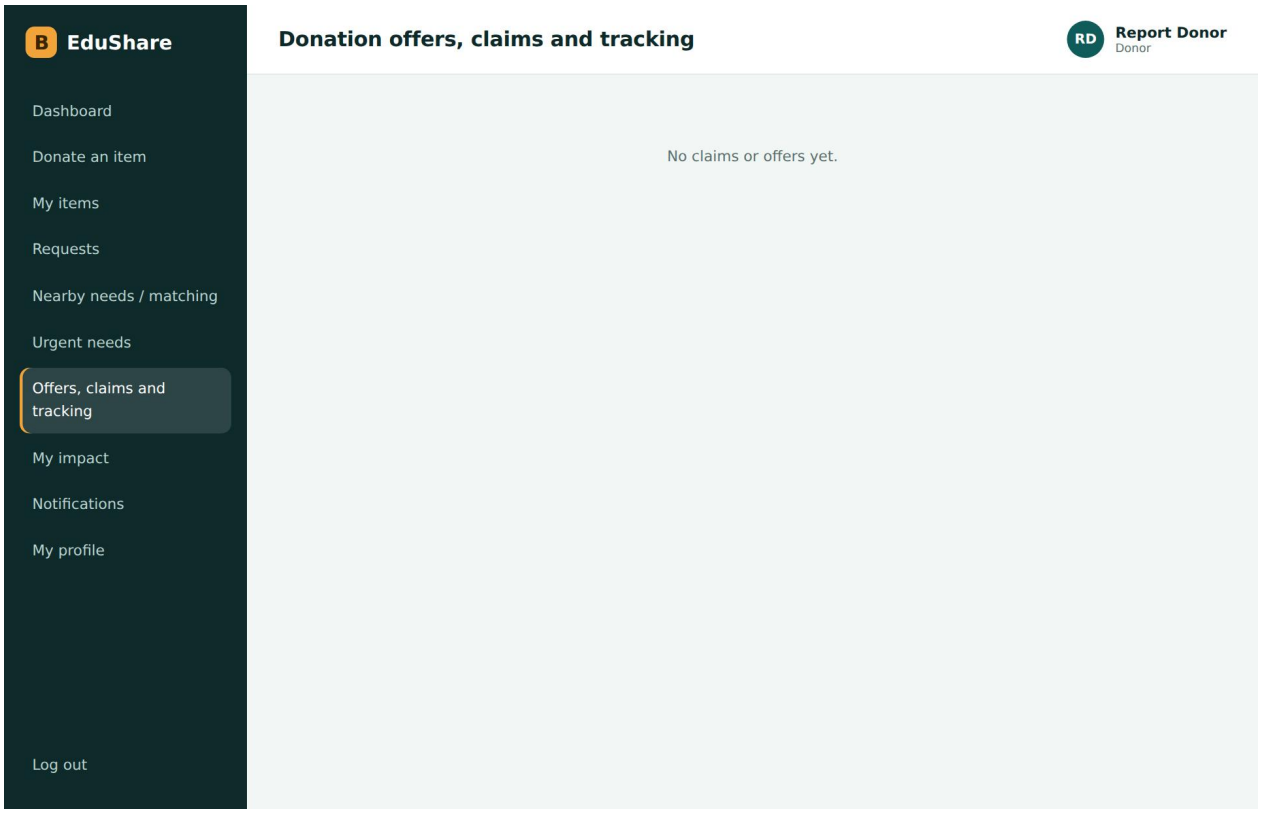


Figure 19. Shared donation offers, claims and tracking page.

Institution dashboard and item search

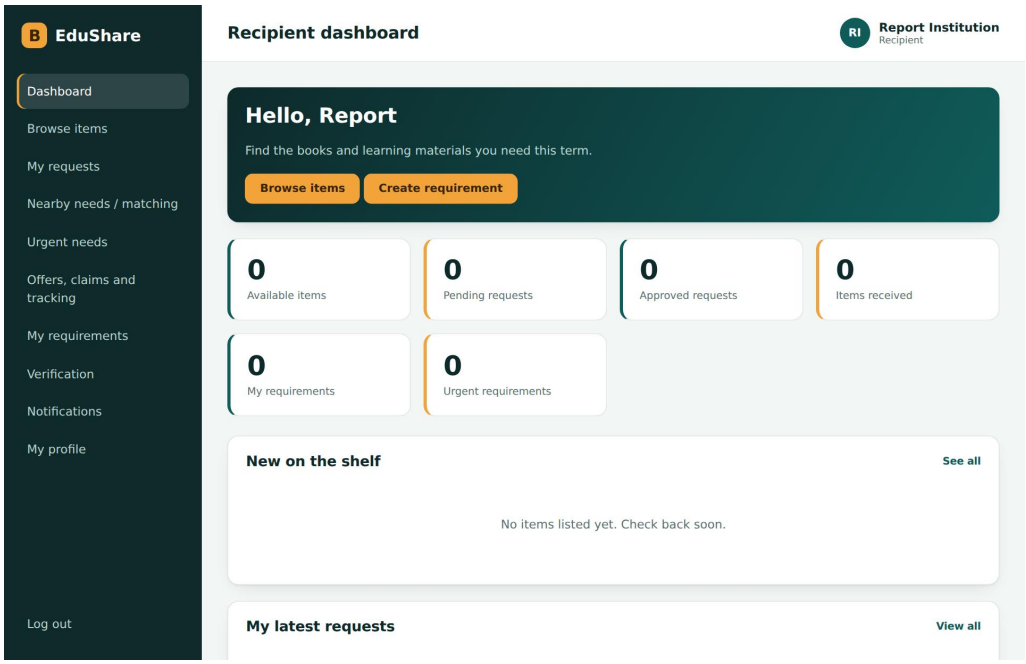


Figure 20. Approved institution dashboard.

An approved institution sees available donations, pending and approved requests, completed receipts and its requirements. Unapproved applicants are redirected to verification.

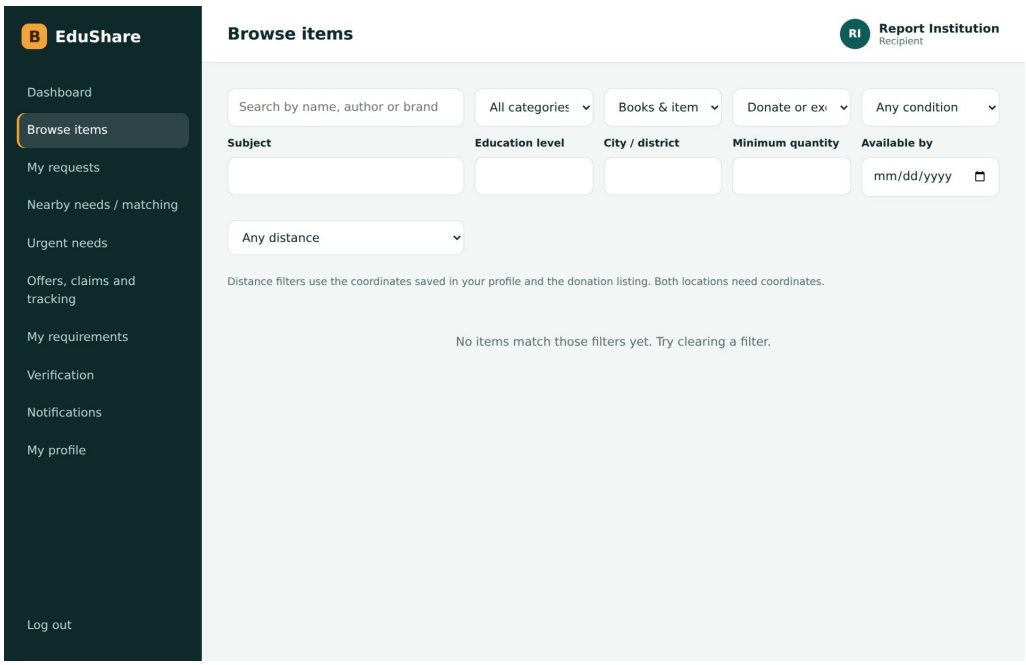


Figure 21. Institution donation browser.

Institutions search and filter available listings and request a permitted quantity. PHP checks institution approval, donor availability and item stock before processing a claim.

Institution requirements

An approved institution records a title, category, subject, grade, quantity, reason and required date. High urgency is reviewed by an administrator before the need appears as approved urgent. Existing requirements can be maintained through the related edit page.

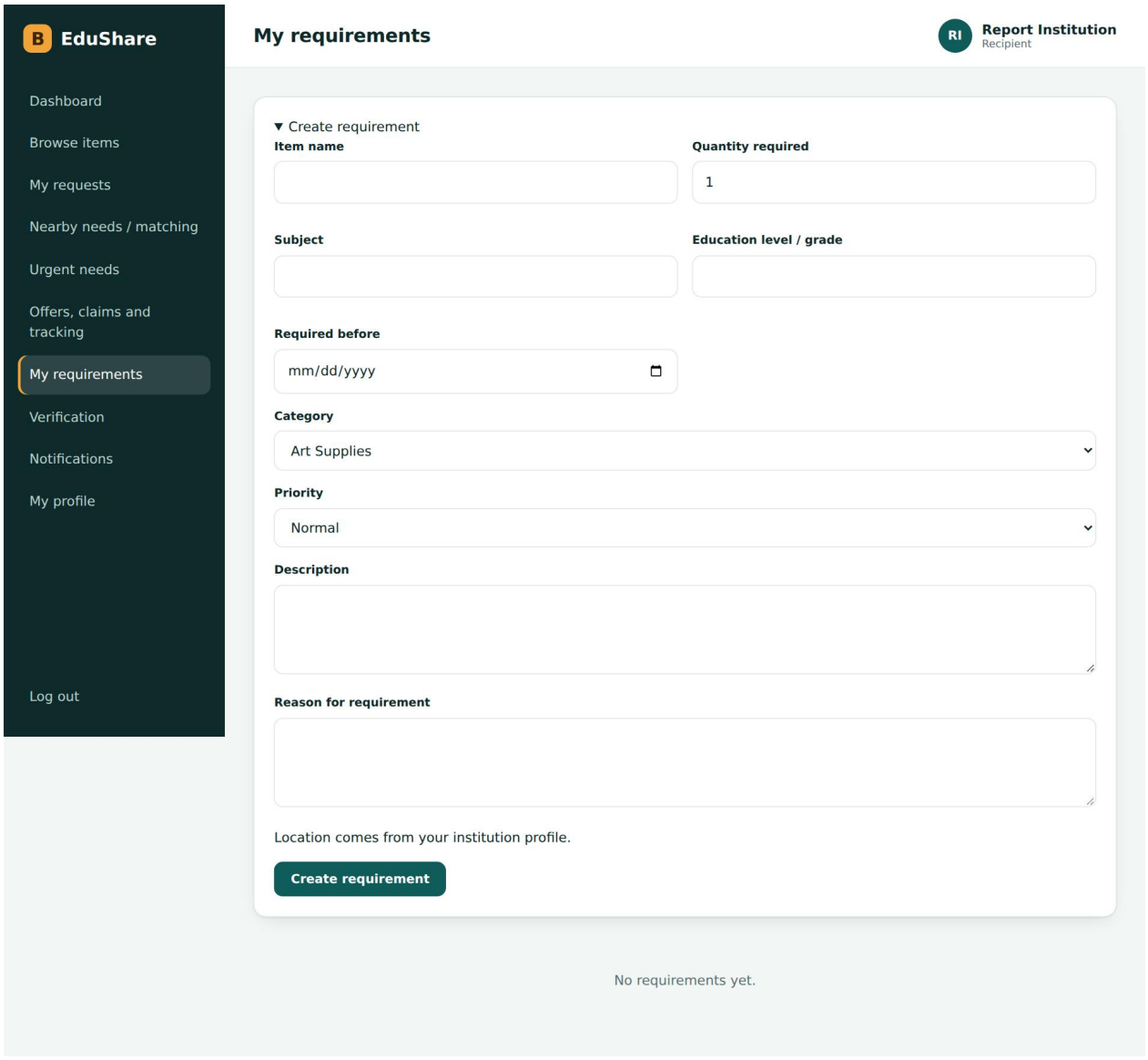


Figure 22. Institution requirement creation form.

Institution verification status

The applicant can view pending, approved, rejected or more-information status and review notes. The preview account is approved solely to capture protected institution interfaces. application.php directs to this verification interface.

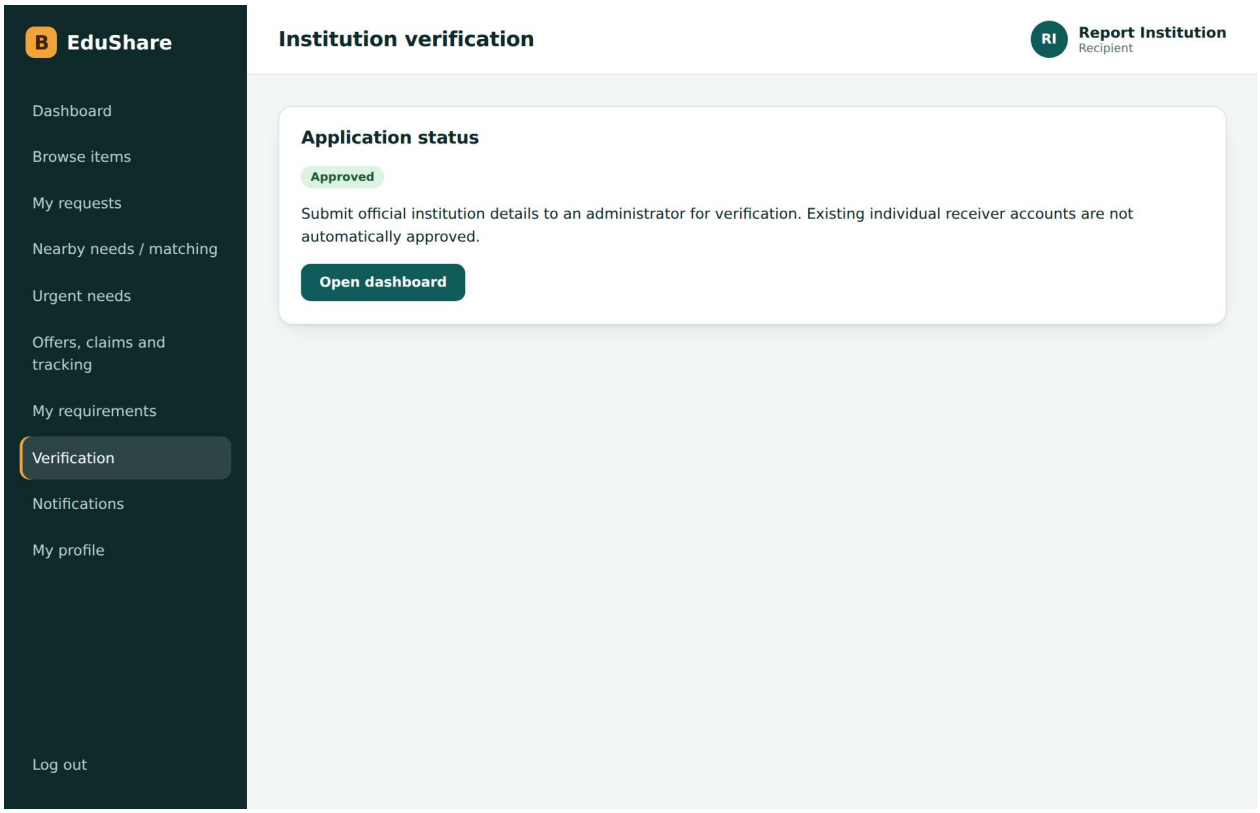


Figure 23. Institution application status page.

Member profile and notifications

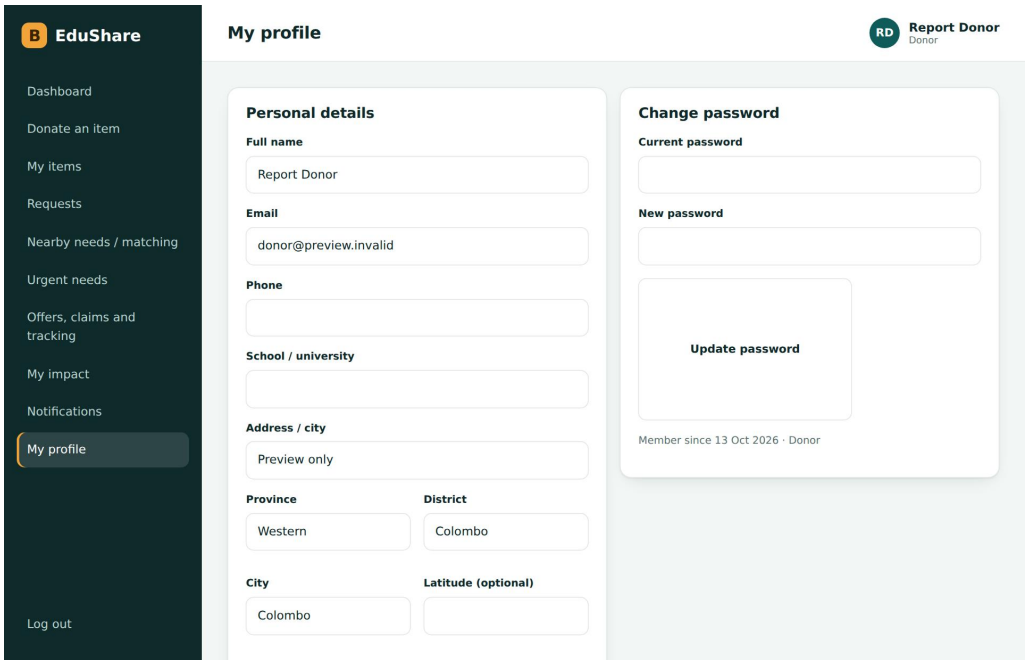


Figure 24. Member profile and password interface.

Users update contact and location details and change their password. Optional coordinates support measured distance filtering; city and district filters remain usable without coordinates.

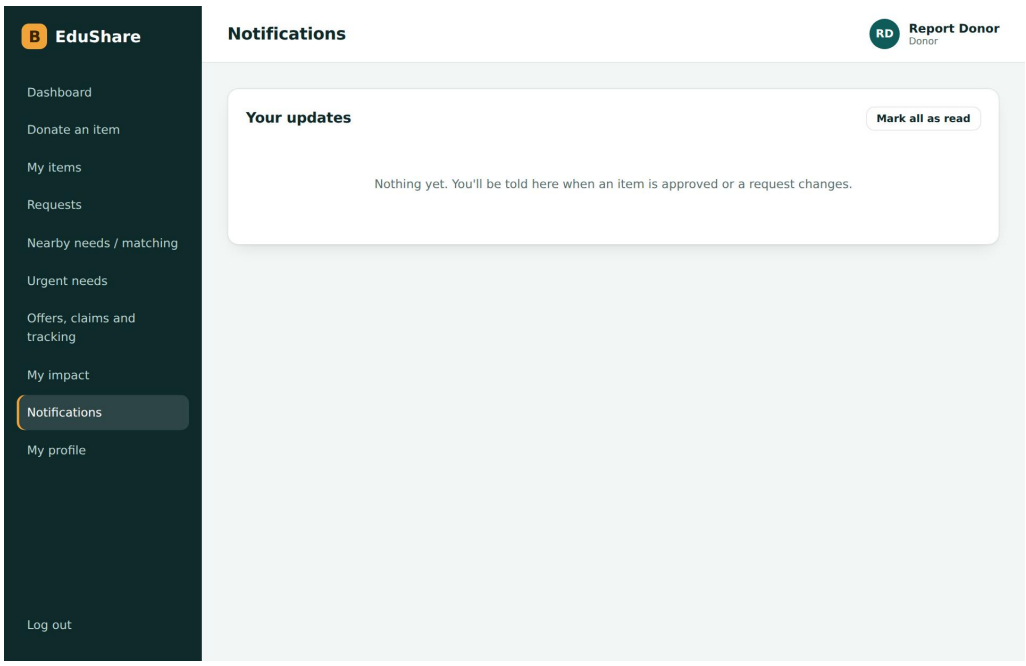


Figure 25. Member notifications interface.

Saved notifications inform members about matching donations, verification and workflow changes. JavaScript polls the notification count for the portal badge, and users can mark notifications as read.

Responsive mobile interface

The mobile capture shows the menu button, hidden sidebar, single-column content and vertically stacked cards. This is the actual CSS breakpoint behaviour, using the same PHP page as the desktop view.

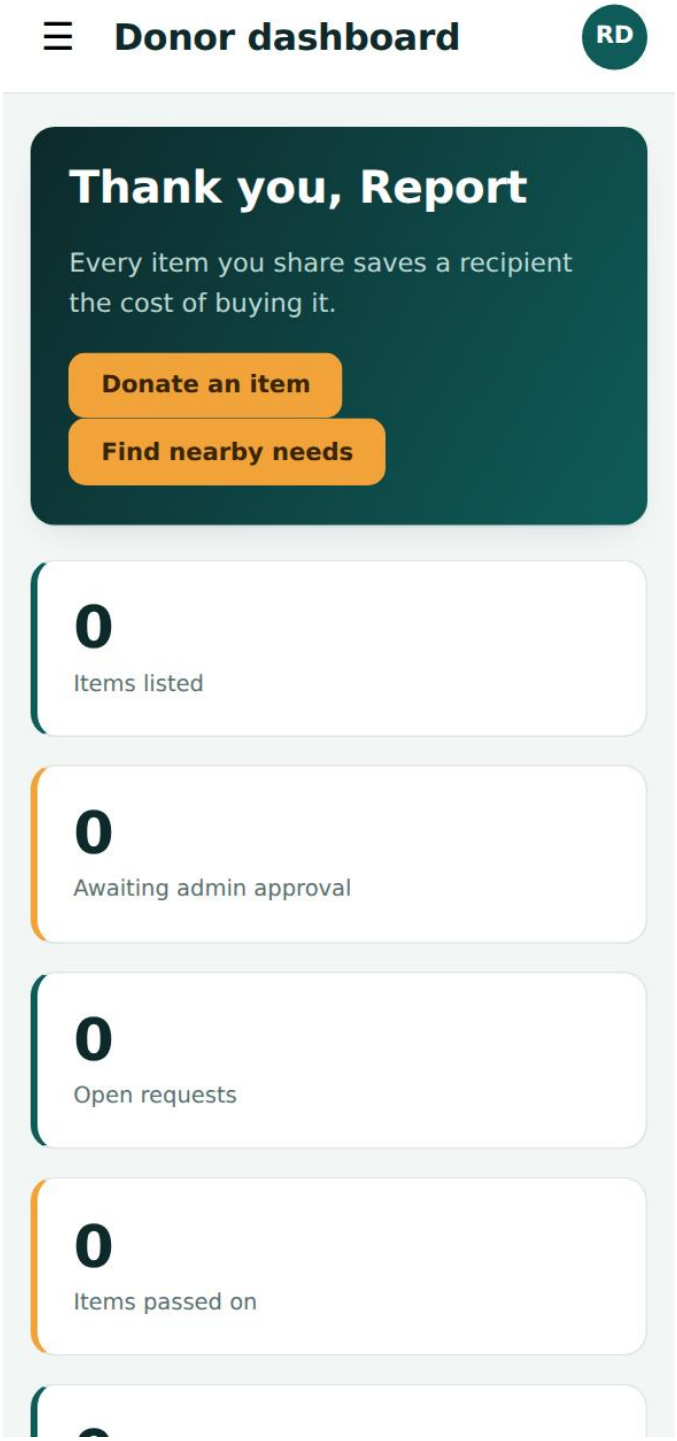


Figure 26. Donor dashboard captured at 390 px viewport width.

Administrator overview and members

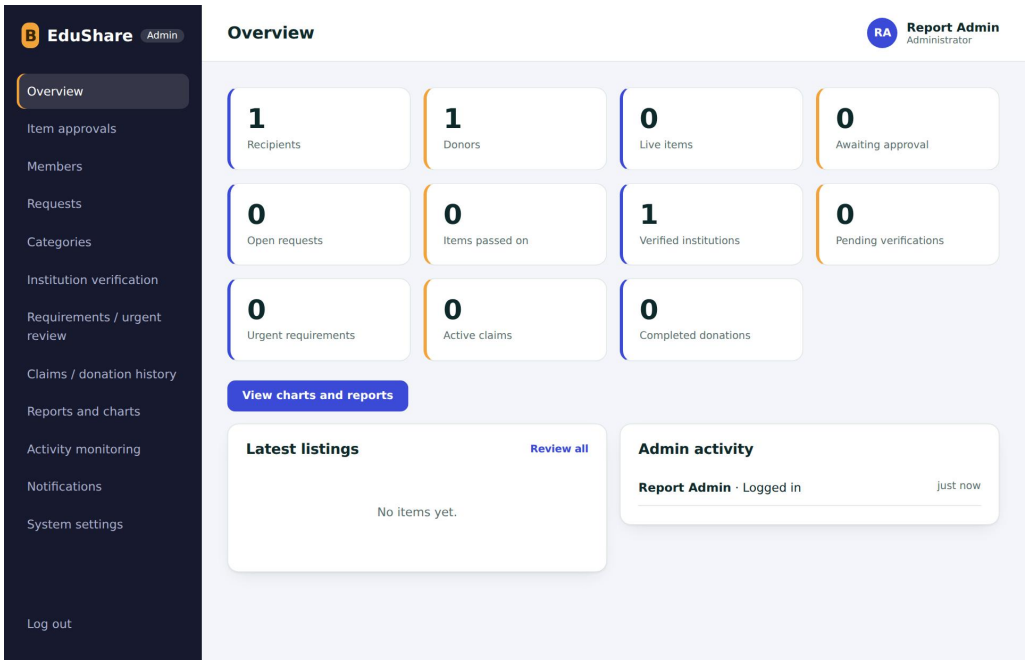


Figure 27. Administrator overview dashboard.

Database totals summarize recipients, donors, listings, verification, active claims and completed donations. Latest listings and administrator activity help staff identify current work.

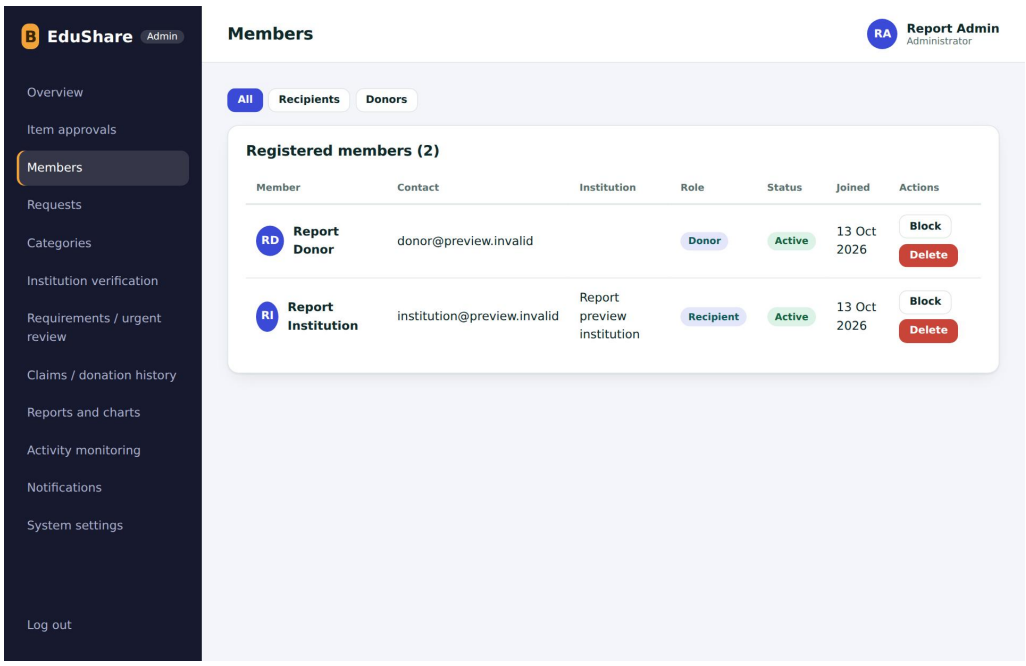


Figure 28. Administrator member management.

Staff can review registered donors and institution representatives and block or restore account access. The two member rows shown are temporary screenshot accounts, not live users.

Administrator listings and requests

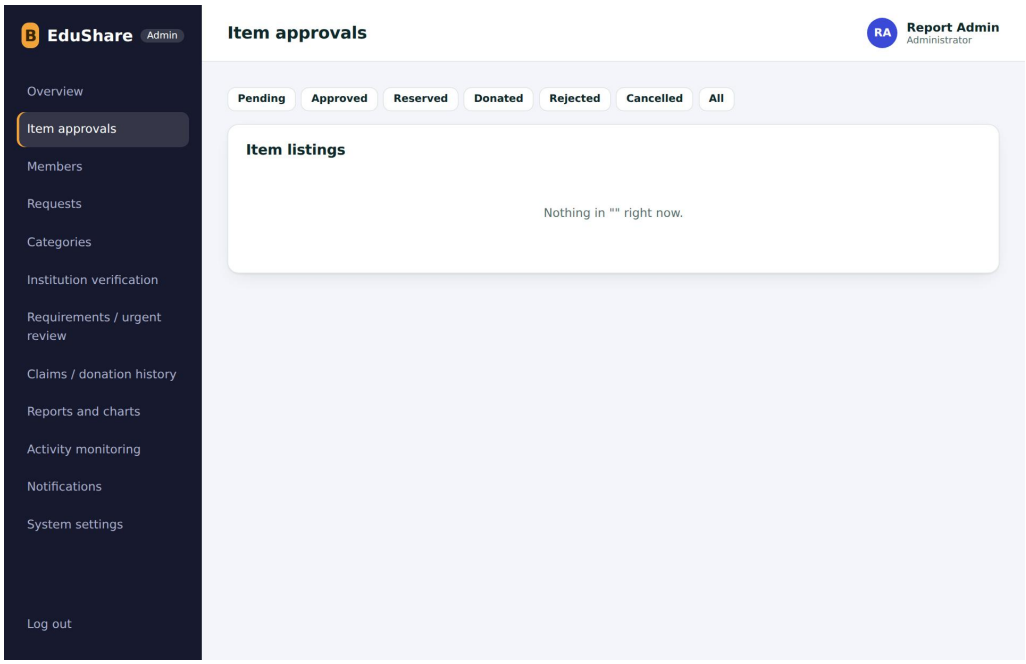


Figure 29. Administrator item management.

Status tabs help staff inspect pending, approved, reserved, donated, rejected and cancelled listings. Staff can moderate suspicious listings; the normal creation path publishes donations immediately.

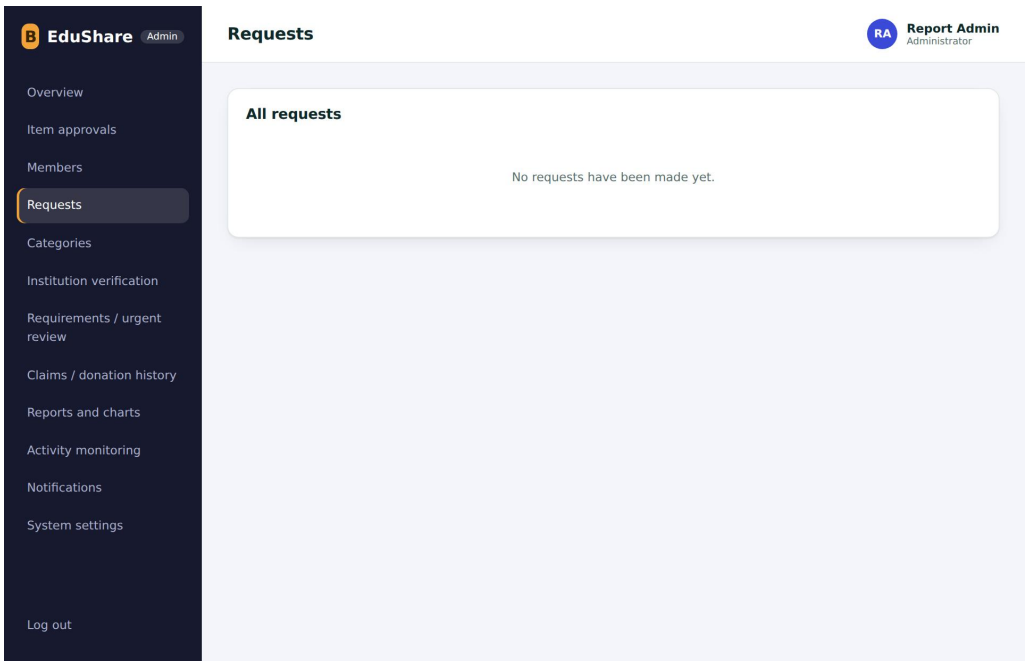


Figure 30. Administrator requests interface.

Staff review donation requests, their parties and recorded status. The requests table also supports donor offers, which appear in the shared claims and donation history interface.

Categories and institution review

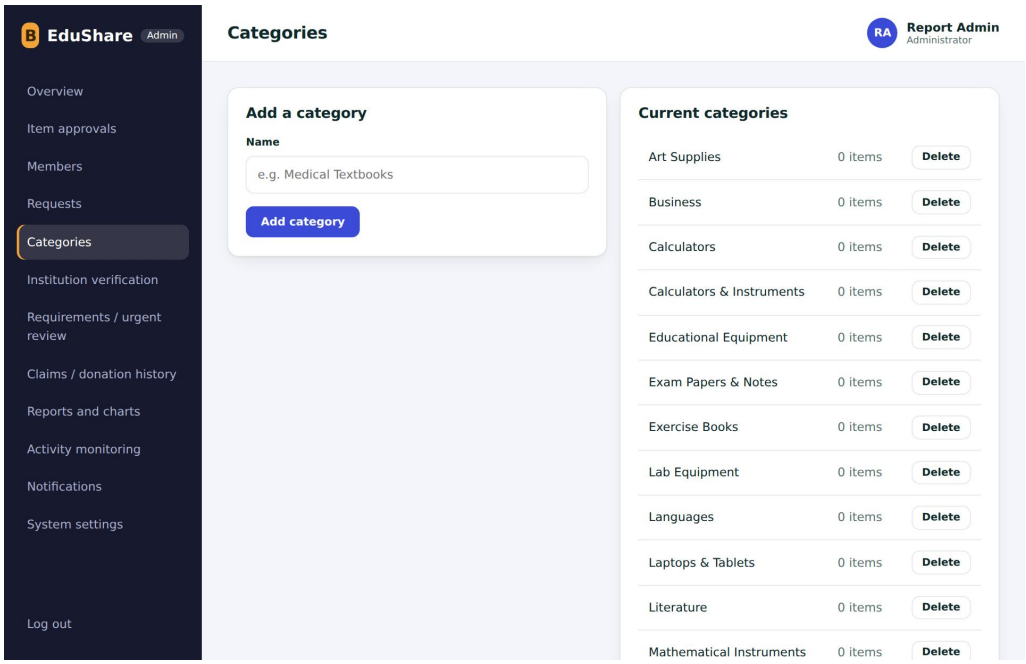


Figure 31. Administrator category management.

Staff maintain shared categories for items and requirements. The displayed categories are the reference rows imported from the SQL files.

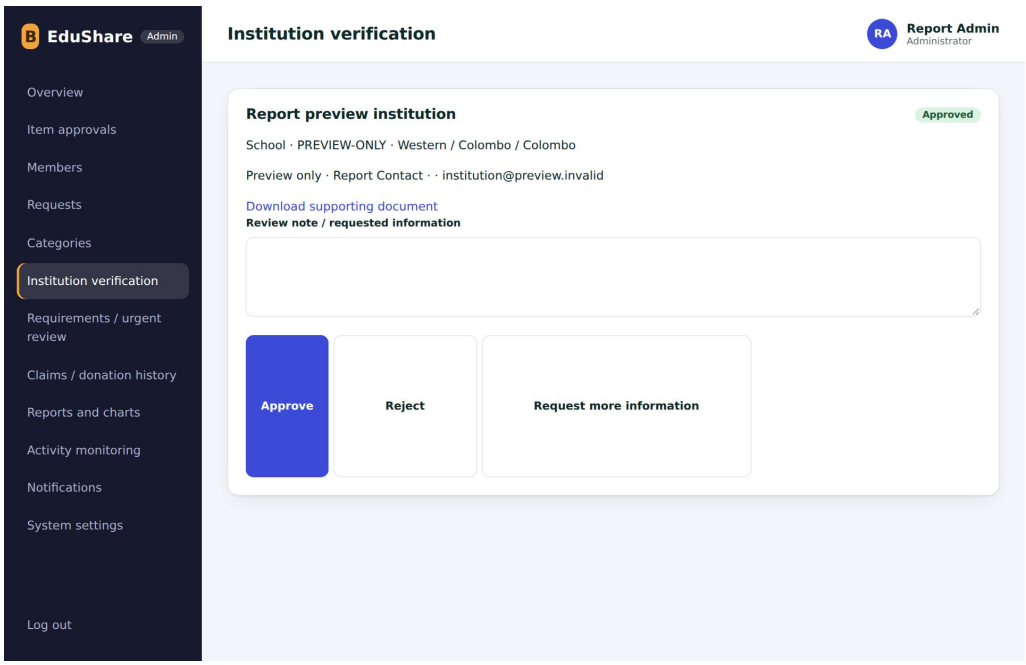


Figure 32. Administrator institution verification form.

Staff inspect application details, download the supporting document and approve, reject or request more information with a review note. PHP updates institutions, sends a member notification and records administrator activity.

Urgency review and reporting

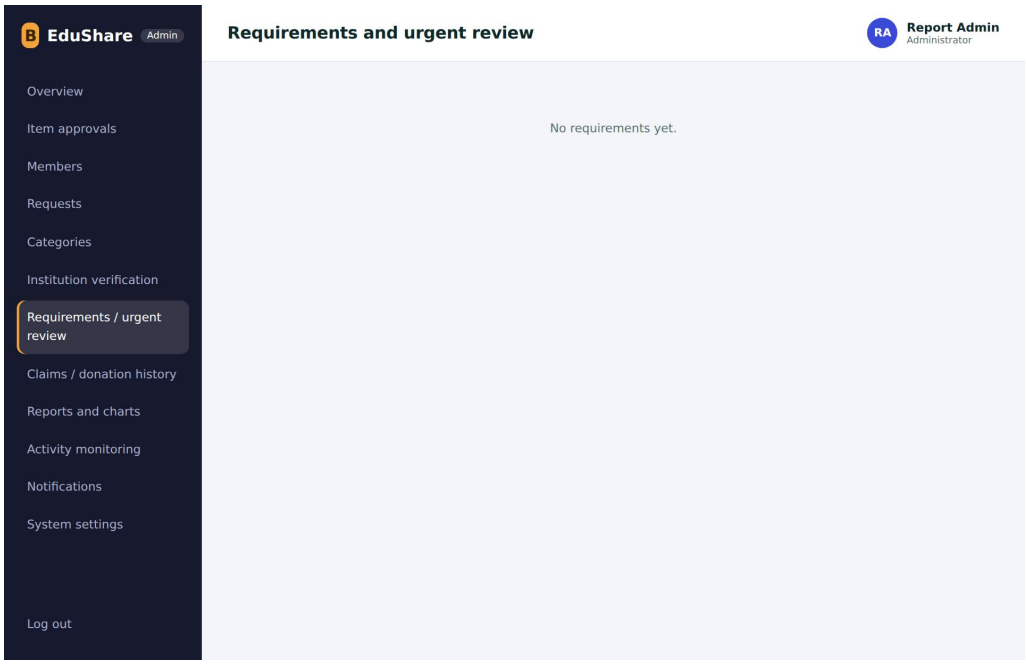


Figure 33. Administrator requirements and urgent review.

Staff review institution requirements and decide whether requested high urgency may be publicly highlighted.

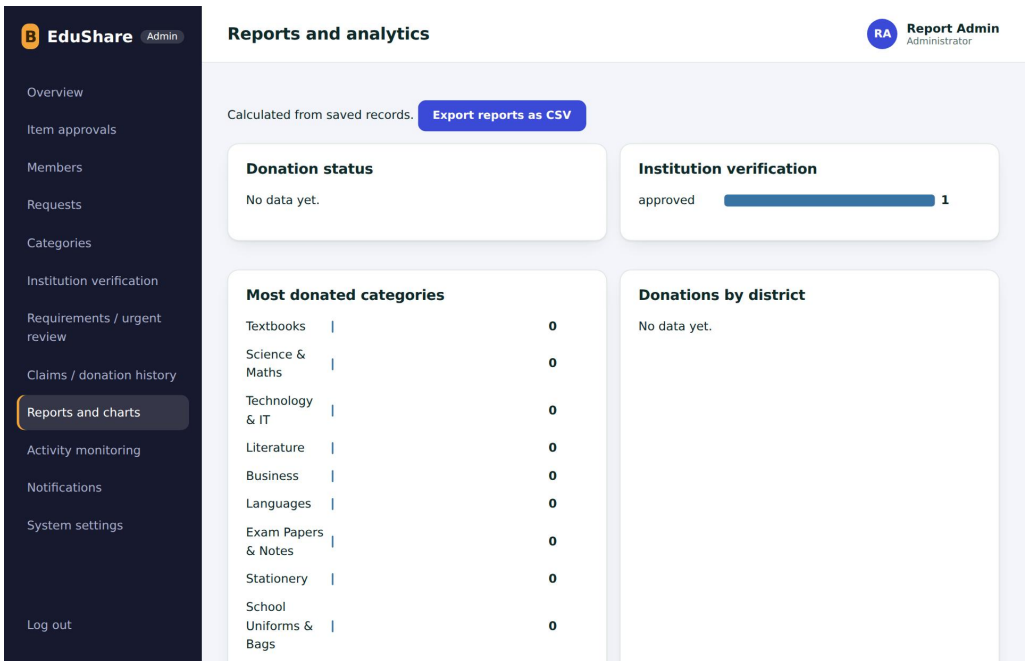


Figure 34. Reports and analytics interface excerpt.

The report page groups saved records by status, category, location, verification, urgency and month. HTML/CSS bars display the totals, and staff can export CSV. This capture shows the upper report cards; additional report groups continue below.

Activity review and admin inbox

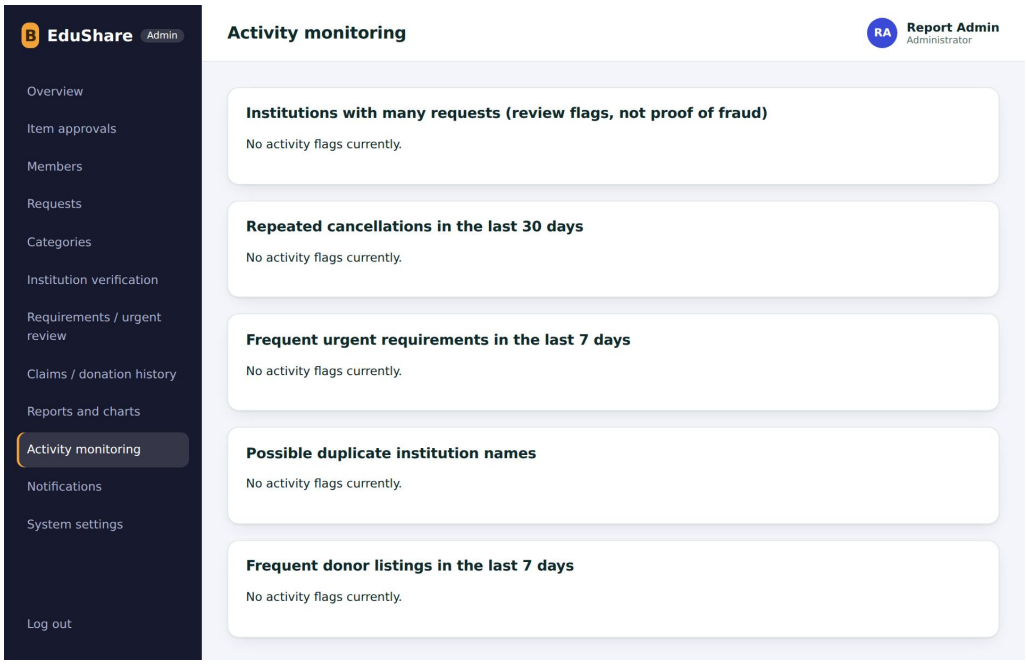


Figure 35. Administrator activity monitoring.

This page highlights frequent requests, repeated cancellations, urgent submissions and possible duplicate institution names. These are review flags for staff and are not automatic findings of fraud.

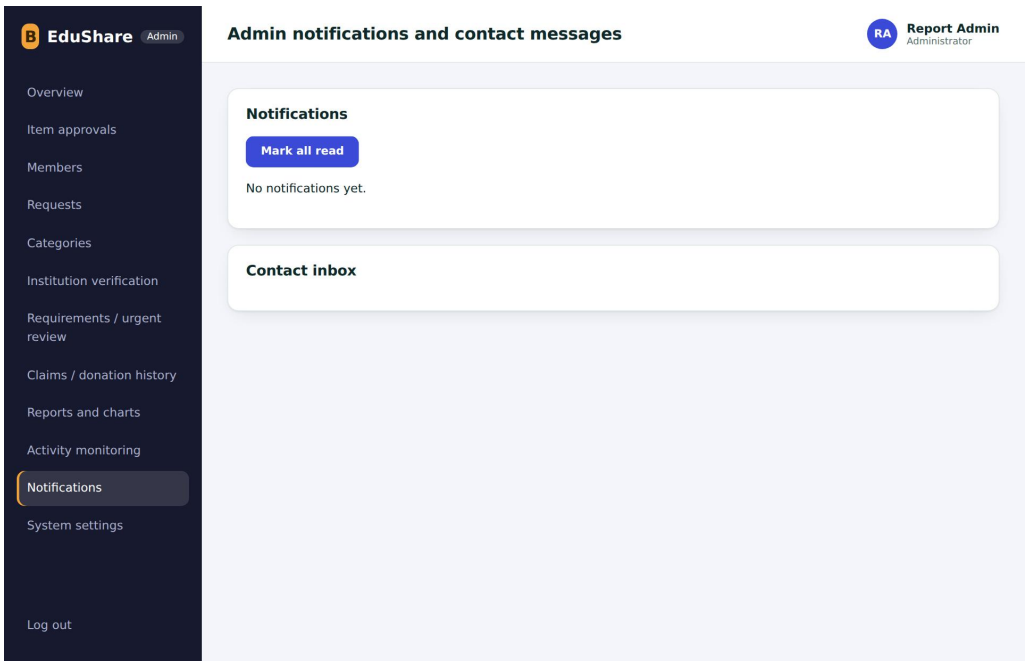


Figure 36. Administrator notifications and contact inbox.

Staff read system notices and messages submitted through the public contact form. Notices can be marked as read, while contact_messages provides the enquiry history.

Administrator settings

The administrator changes their password by supplying the current password and matching new values of at least ten characters. Database settings and the base URL are configured in includes/config.php.

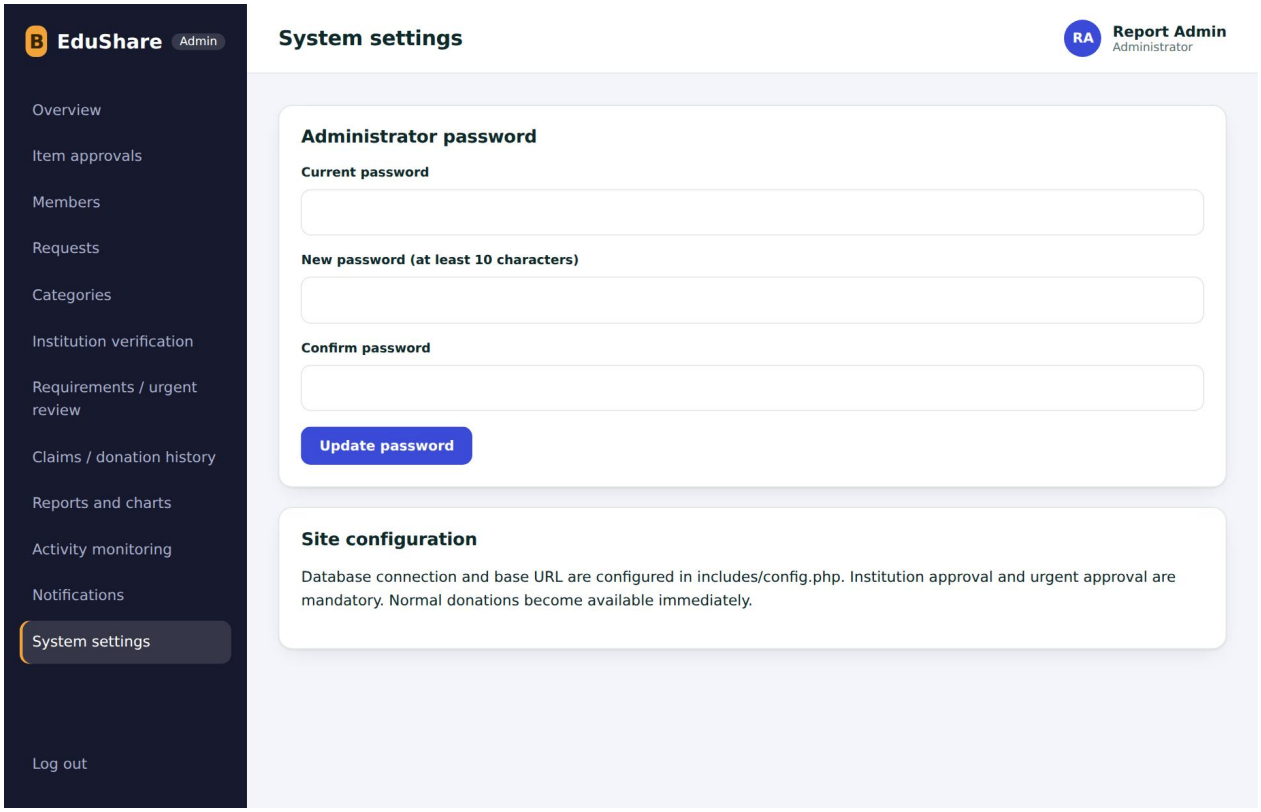


Figure 37. Administrator password and configuration interface.

5. Future enhancements

5.1 Improvements to the current system

Proposal	Expected improvement
Email and account recovery	Verify email ownership, add password reset and deliver important notifications outside the portal.
Stronger verification	Connect with authorized institutional registries, improve duplicate detection and record more structured review evidence.
Data integrity and auditability	Add a declared requirement foreign key where appropriate, use structured actor IDs for history and review constraints and retention rules.
Accessibility and mobile review	Improve labels, focus states, screen-reader feedback and navigation; test all pages on multiple browsers and screen sizes.
Operational readiness	Use HTTPS, production secrets, controlled upload access, backups, monitoring and a repeatable deployment procedure.

5.2 Additional features

Useful additions include email or SMS reminders, saved searches, institution feedback after receipt, a clearly defined reciprocal exchange workflow and optional logistics integration. A map could support collection planning, subject to approval for any external map service. Matching could be improved by measuring successful matches before adding more complex recommendations.

5.3 Commercialization and scalability

EduShare could be operated with schools, educational NGOs and corporate social responsibility programmes. A future model could keep core donations free while funding hosting through sponsorship or optional reporting services. This is a proposed model; no pricing, revenue or commercial partners are implemented.

For growth, add pagination and query indexes, optimize searches, separate uploaded files from application servers and introduce background notification processing. Load testing, recovery testing and agreed data protection practices should precede a wider launch.

6. Deployment and submission preparation

6.1 Local installation

1. Place the edushare folder inside XAMPP htdocs and start Apache and MySQL.

2. Check BASE_URL, database host, user and password in includes/config.php.

3. For a fresh installation, run setup.php, which imports database.sql and upgrade.sql and creates an administrator with chosen credentials. Remove setup.php afterwards.

4. For an existing installation, back up both databases and uploaded files, then apply upgrade.sql according to README.txt. Preserve existing connection settings and private verification documents.

5. Register a real donor and institution, review the institution as administrator, then demonstrate a listing, claim or offer, acceptance and receipt confirmation.

6.2 Verification performed for this report

The base SQL and upgrade SQL imported successfully into the isolated preview database. A browser captured 37 application views and three read-only database inspection views. The screenshot script checked the returned page text for PHP warnings and fatal errors. These checks confirm page rendering for the captured empty-data states; they do not constitute full workflow, security or production testing.

6.3 Submission checklist

Complete the Group ID, full names, index numbers and degree programme for all four members; confirm the proposed contribution allocation. Include this PDF and the complete source code in a single ZIP named with the Group ID, such as 01.zip. The group leader submits the one allowed group submission through the LMS. Every member must attend the viva and understand the entire system, as specified in the assignment guidelines.

7. Conclusion and source references

Conclusion

EduShare provides a database-backed platform for sharing books and educational materials with verified institutions. The supplied implementation includes role-based portals, listings, requirements, matching, quantity-aware claims and offers, notifications, a delivery workflow and administrator reporting. Its structure demonstrates how PHP connects the frontend to relational records and enforces the main donation rules.

The report covers the required overview, technologies, database design, interfaces and future enhancements.

Source references

Source	Use in this report
sql/database.sql and sql/upgrade.sql	Database entities, fields, keys, relationships and migration behaviour.
includes/config.php, functions.php and extensions.php	PDO connections, sessions, access checks, uploads, distance calculations, quantity allocation and completion.
assets/css/style.css and assets/js/app.js	Responsive layouts, navigation, live search, form behaviour and notification polling.
Public, donor, recipient and admin PHP pages	Page functionality, forms, database queries, reports and workflow interface explanations.